

Resumen

La inteligencia artificial se ha convertido en el campo de la informática con mayor ritmo de evolución y uno de los que más impacto global está generando, con una velocidad de desarrollo pocas veces vista en la historia tecnológica. Su utilidad potencial es enorme, y sus aplicaciones abarcan desde el ámbito industrial y médico hasta el social y educativo. Las soluciones basadas en IA permiten automatizar tareas complejas, aumentar la eficiencia de los procesos y abrir nuevas oportunidades en sectores donde antes no se contemplaba el uso de estas tecnologías.

Este trabajo consiste en el desarrollo completo de una aplicación web que será utilizada por los trabajadores de la ONG ADIMA, dedicada a la atención de menores. La herramienta les permitirá subir sus grabaciones de audio y obtener una transcripción automática generada por un modelo de inteligencia artificial. Además, la aplicación ofrece una interfaz intuitiva para revisar y editar esas transcripciones antes de exportarlas en distintos formatos (como TXT o DOCX), lo que facilita su uso documental en entornos judiciales. El desarrollo incluye funcionalidades clave como el control de usuarios, la gestión de archivos, la edición de texto con marcas temporales y la exportación personalizada de los resultados.

Cabe destacar que no se trata de un proyecto conceptual o experimental: la aplicación ha sido diseñada para un uso real, y ha sido presentada y validada por los propios trabajadores de la ONG, quienes han mostrado una valoración muy positiva del sistema. La utilidad práctica de la herramienta, su diseño adaptado al flujo de trabajo de la entidad y el tratamiento local de los datos (que garantiza la confidencialidad de la información) han sido elementos especialmente valorados. Esta validación directa por parte de los usuarios finales refuerza el valor social y funcional del proyecto.

Previo al desarrollo técnico, se realizó un estudio sobre la viabilidad del modelo Whisper en sus diferentes variantes, con el objetivo de determinar cuál ofrece el mejor equilibrio entre precisión, velocidad de inferencia y facilidad de integración. Asimismo, se analizaron las diferencias de rendimiento al ejecutar estos modelos en CPUs frente a GPUs, valorando cómo la disponibilidad de hardware influye en los tiempos de procesamiento y en la experiencia de uso.

Palabras clave: IA, inteligencia artificial, Whisper, OpenAI, ASR.

Abstract

Artificial intelligence has become the fastest-evolving field in computer science and one of the areas generating the greatest global impact, with a pace of development rarely seen in technological history. Its potential utility is enormous, with applications ranging from industrial and medical domains to social and educational sectors. AI-based solutions enable the automation of complex tasks, increase process efficiency, and open new opportunities in fields where the use of such technologies was previously unconsidered.

This work consists of the complete development of a web application to be used by the workers of the NGO ADIMA, dedicated to the care of minors. The tool will allow them to upload their audio recordings and obtain an automatic transcription generated by an artificial intelligence model. Furthermore, the application offers an intuitive interface to review and edit these transcriptions before exporting them in various formats (such as TXT or DOCX), facilitating their documentary use in judicial environments. The development includes key features such as user control, file management, text editing with timestamps, and customized export of results.

It is worth noting that this is not a conceptual or experimental project: the application has been designed for real use and has been presented and validated by the NGO's own workers, who have expressed a very positive evaluation of the system. The practical usefulness of the tool, its design adapted to the organization's workflow, and the local handling of data (which guarantees information confidentiality) have been especially appreciated. This direct validation by end users reinforces the social and functional value of the project.

Prior to the technical development, a study was conducted on the feasibility of the Whisper model in its different variants, aiming to determine which offers the best balance between accuracy, inference speed, and ease of integration. Likewise, performance differences were analyzed when running these models on CPUs versus GPUs, assessing how hardware availability influences processing times and user experience.

Keywords: AI, artificial intelligence, Whisper, OpenAI, ASR.

Índice general

1	Introducción y motivación	1
1.1.	Contexto del TFG	1
1.1.1.	Introducción al contenido general del TFG	1
1.1.2.	Objetivos del TFG	5
1.1.3.	Alcance del TFG	7
1.1.4.	Grado de dificultad	7
1.1.5.	Grado de cumplimiento de los objetivos planteados	8
1.1.6.	Potencial aplicación de este trabajo en un entorno académico o industrial	9
1.2.	Estructura de la memoria	10
2	Gestión del proyecto	12
2.1.	Introducción	12
2.2.	Planificación estimada	13
2.3.	Gestión del coste	15
2.3.1.	Costes directos	15
2.3.2.	Costes indirectos	16
2.4.	Gestión de riesgos	17
2.4.1.	Riesgo individual del proyecto	19
2.4.2.	Riesgo general del proyecto	19
2.4.3.	Matriz de riesgos	19
2.5.	Metodología	21
2.6.	Conclusión	22
3	Análisis del problema	24
3.1.	Introducción	24
3.2.	Requisitos de información	24
3.3.	Requisitos funcionales	25
3.4.	Requisitos no funcionales	26
3.5.	Conclusiones	27
4	Diseño de la solución	28
4.1.	Introducción	28
4.2.	Arquitectura global	28
4.2.1.	Frontend	29
4.2.2.	Backend	33
4.2.3.	Modelo	35
4.2.4.	Base de datos	36
4.2.5.	Gestión de colas	38

5 Implementación	39
5.1. Introducción	39
5.2. Configuración del entorno de desarrollo	39
5.3. Pruebas del modelo ASR	39
5.3.1. Experimentación	40
5.3.2. Conclusiones	43
5.4. Desarrollo del frontend	43
5.4.1. Pantalla de login	43
5.4.2. Pantalla de bienvenida	44
5.4.3. Pantalla de transcripción	48
5.5. Desarrollo del backend	50
5.5.1. Modelos	50
5.5.2. Serializadores	51
5.5.3. Endpoints y estructura de la API REST	53
5.5.4. Autenticación y permisos	55
5.5.5. Exportación de transcripciones	56
5.6. Gestión de colas	56
5.7. Futuras extensiones	58
5.7.1. Fine-tuning de Whisper	58
5.7.2. Infraestructura de cómputo dedicada	58
5.7.3. Estimación de presupuesto	59
5.7.4. Dockerización y despliegue	59
6 Conclusiones	61
Bibliografía	63

Índice de figuras

2.1. Diagrama de Gantt del proyecto	14
2.2. Actividades del proyecto	15
4.1. Mockup de pantalla de inicio de sesión	30
4.2. Mockup de pantalla de bienvenida	30
4.3. Mockup de pantalla de transcripción	31
4.4. Primer diseño UML de la base de datos	37
5.1. Pagina de inicio de sesión	44
5.2. Pantalla de bienvenida	45
5.3. Pantalla de bienvenida con archivo preparado para subir	46
5.4. Pantalla de bienvenida con menú para crear carpeta	46
5.5. Pantalla de bienvenida con menú para eliminar carpeta	47
5.6. Pantalla de bienvenida con menú para añadir etiquetas	47
5.7. Pantalla de bienvenida con menú para añadir etiquetas completo	48
5.8. Pantalla de transcripción	49
5.10. Pantalla de transcripción con menú de exportación	49
5.9. Pantalla de transcripción con checkmarks	50
5.11. Diagrama de clases	52

Índice de tablas

2.1. Matriz de evaluación de riesgos	20
5.1. Tiempos de transcripción con Whisper en el ordenador menos potente	41
5.2. Tiempos de transcripción con WhisperX en el ordenador menos potente	41
5.3. Tiempos de transcripción con Whisper en el ordenador más potente .	42
5.4. Tiempos de transcripción con WhisperX en el ordenador más potente	42

Índice de extractos de código

1. Introducción y motivación

1.1. Contexto del TFG

La ONG Adima se encarga del peritaje judicial en casos de abuso a niños. Sus trabajadores entrevistan de forma sutil y poco intrusiva a los niños y obtienen grabaciones. Por último, estas grabaciones se transcriben para obtener los testimonios y son utilizados en el juzgado para evaluar los casos. [1]

El problema nace cuando los trabajadores se dan cuenta de que tienen que invertir una enorme porción de su jornada laboral en realizar dichas transcripciones a mano. La solución podría estar en utilizar servicios en línea para hacer transcripciones, pero dicha opción es imposible dado a la sensibilidad de los datos.

En el desarrollo de la aplicación, la privacidad y el tratamiento local de datos sensibles han sido aspectos muy importantes. Esto se debe a que el contenido de los audios tratados corresponde a entrevistas entre los trabajadores de la ONG y personas en situación de vulnerabilidad. En estos audios puede haber testimonios delicados, por lo que es necesario asegurar que no se expongan a riesgos fuera de la red local. Por ello, hemos decidido prescindir de servicios en la nube o transferencias a servidores externos. Esto implica que cualquier fallo en el servidor local podría hacer que se perdieran los audios si no se hacen las copias de seguridad adecuadas, pero es un riesgo aceptable para asegurar que los audios se mantienen privados.

1.1.1. Introducción al contenido general del TFG

La inteligencia artificial (IA) es una disciplina de la informática que busca crear sistemas capaces de realizar tareas que normalmente requieren la inteligencia humana. En otras palabras, se diseña tecnología que permite a las máquinas aprender, razonar o resolver problemas complejos de forma autónoma. Gracias a los avances en aprendizaje automático y redes neuronales profundas, hoy la IA puede ejecutar muchas funciones cognitivas: por ejemplo, aprender de la experiencia, adaptarse a nuevas entradas y ejecutar tareas humanas como el reconocimiento de voz, la toma de decisiones, la traducción de idiomas o la visión por computador. Entre las aplicaciones típicas de la IA se cuentan, por ejemplo, asistentes de voz inteligentes, diagnóstico médico asistido por computadora, vehículos autónomos o sistemas de recomendación.

Redes neuronales artificiales. Las redes neuronales artificiales (ANN, del inglés Artificial Neural Networks) son uno de los modelos más importantes en aprendizaje automático, inspirados en la estructura del cerebro humano. [2] Una ANN está formada por unidades de cálculo interconectadas llamadas neuronas artificiales. Cada neurona recibe señales de entrada (números reales), las combina

linealmente y aplica una función no lineal (función de activación) para producir una salida. Las conexiones entre neuronas llevan asociados pesos, que determinan la fuerza de la señal que se transmite, y estos pesos se ajustan durante el proceso de entrenamiento. En la práctica, las neuronas se organizan en capas: una capa de entrada que recibe los datos brutos, una o más capas ocultas que procesan la información y una capa de salida que genera la predicción o respuesta final. Los datos fluyen hacia adelante a través de estas capas (de la entrada a la salida), y en redes profundas (deep learning) hay múltiples capas ocultas. Esta estructura jerárquica permite a la red aprender representaciones progresivamente más complejas.

Por ejemplo, en una red neuronal para visión por computador, la capa de entrada podría recibir un vector de características de una imagen, las capas ocultas procesan patrones complejos y la capa de salida entrega la etiqueta predicha ("perro" o "gato"). [3] El aprendizaje de la red ocurre ajustando los pesos de cada conexión según los errores cometidos en las predicciones.

Entrenamiento e inferencia de modelos. El entrenamiento de una red neuronal es el proceso iterativo por el cual el modelo aprende de los datos. Se presenta al modelo un conjunto de ejemplos etiquetados (entrada y salida deseada) y se mide el error de la predicción. Luego se aplica retropropagación: se calcula el gradiente del error respecto a cada peso y se actualizan los pesos para reducir la pérdida. En cada iteración (o época), la red realiza un pase hacia adelante (calcula la salida) y un pase hacia atrás (ajusta los pesos). Este ciclo se repite hasta que el modelo converja o alcance la precisión deseada. En resumen, el entrenamiento implica optimizar los parámetros de la red mediante algoritmos como el descenso de gradiente.

En contraste, la inferencia es la fase en la que el modelo ya entrenado se utiliza para hacer predicciones con datos nuevos. [4] Durante la inferencia, los pesos permanecen fijos y el modelo procesa la señal de entrada para generar su salida sin realizar más ajustes. Es decir, entrenamiento es el proceso de optimización del modelo, mientras que inferencia es su uso operativo posterior. Por ejemplo:

- **Entrenamiento:** Ajuste de los pesos del modelo con un conjunto de datos de entrenamiento.
- **Inferencia:** Aplicación del modelo entrenado para generar predicciones sobre nuevos datos.

La inferencia suele requerir menos recursos computacionales que el entrenamiento, ya que se limita a ejecutar operaciones fijas para producir la salida deseada (por ejemplo, multiplicaciones de matrices).

Fine-tuning (ajuste fino). El fine-tuning es una técnica relacionada con el entrenamiento que consiste en adaptar un modelo previamente entrenado a una nueva tarea o conjunto de datos. [5] En lugar de entrenar una red neuronal desde cero, se toma un modelo base (preentrenado con datos generales) y se continúa su entrenamiento usando datos específicos del dominio de interés. De este modo se aprovechan los conocimientos previos del modelo, logrando mejoras en tareas especializadas de forma más rápida y con menos datos etiquetados.

Este enfoque es especialmente útil en los modelos grandes: entrenar de cero redes con millones (o miles de millones) de parámetros es costoso, pero ajustar un modelo ya entrenado suele ser mucho más económico. Por ejemplo, se puede tomar un modelo general de reconocimiento de voz y hacer fine-tuning con grabaciones de un contexto específico (médico, legal, técnico) para mejorar su desempeño en ese ámbito sin empezar el entrenamiento desde cero.

Modelos ligeros vs modelos grandes. Los modelos de IA se distinguen también por su tamaño (número de parámetros). Un modelo ligero contiene relativamente pocos parámetros y está optimizado para eficiencia y velocidad. En cambio, un modelo grande tiene un número muy elevado de parámetros (centenas de millones o incluso miles de millones) y ofrece mayor capacidad de representación a costa de mayor complejidad computacional. Por ejemplo, la familia de modelos Whisper de OpenAI va desde la variante tiny, con unos 39 millones de parámetros, hasta la variante large, con aproximadamente 1.550 millones de parámetros.

- **Modelos ligeros:** contienen pocos parámetros (39 M en Whisper tiny). Ventajas: requieren menos memoria y se inferen rápidamente, incluso en hardware limitado. Desventajas: pueden ser menos precisos en tareas complejas.
- **Modelos grandes:** contienen muchos parámetros (1.550 M en Whisper large). Ventajas: mayor precisión y capacidad para reconocer patrones complejos. Desventajas: requieren mucho más tiempo y recursos de cómputo para entrenar e inferir

Esta diferencia se traslada directamente a la práctica: un modelo ligero podrá ejecutarse en dispositivos con recursos limitados o procesar audio en tiempo real en una CPU, mientras que un modelo grande puede necesitar GPU potentes o sistemas de alto rendimiento para funcionar eficientemente.

Inferencia en CPU vs GPU. La infraestructura de cómputo afecta notablemente el rendimiento de la inferencia. Las CPUs (unidades centrales de proceso) están optimizadas para tareas de procesamiento secuencial y manejo de diversos tipos de operaciones. Tienen relativamente pocos núcleos potentes y funcionan bien en cargas generales de trabajo. Por ello, una CPU puede manejar inferencia de modelos ligeros o realizar preprocesamiento de datos sin problemas. En cambio, las GPUs (unidades de procesamiento gráfico) cuentan con miles de núcleos más simples diseñados para procesamiento en paralelo masivo. Esta arquitectura las hace ideales para las operaciones matriciales intensivas que requieren los modelos de redes neuronales profundas.

En concreto, las GPUs aceleran enormemente el cálculo de multiplicaciones de matrices y algoritmos de optimización, reduciendo drásticamente el tiempo de entrenamiento e inferencia en comparación con las CPUs. [6] En GPU, se puede procesar simultáneamente grandes bloques de datos, lo que es crítico para inferencia rápida de modelos complejos. Por ejemplo, en aprendizaje profundo las GPUs pueden acelerar la multiplicación de matrices y la retropropagación, reduciendo mucho los tiempos de entrenamiento y mejorando la velocidad de

inferencia. En CPU, el procesamiento es más lento para cargas paralelas intensivas, pero las CPUs consumen menos energía y son más comunes en equipos de consumo. Son adecuadas para inferencia de modelos pequeños o cuando se dispone de restricciones de hardware. Las CPUs manejan bien tareas secuenciales y con estructuras de memoria aleatorias.

En resumen, escoger entre CPU y GPU depende del balance deseado entre velocidad y disponibilidad de recursos: las GPUs ofrecen mayor velocidad para modelos grandes y paralelismo, mientras que las CPUs bastan para tareas más ligeras o cuando no se dispone de GPUs.

Reconocimiento Automático del Habla (ASR). El reconocimiento automático del habla (ASR) es la tecnología que permite convertir el habla humana en texto escrito. [?] También se le conoce como “conversión de voz a texto”. Los sistemas ASR modernos utilizan IA y aprendizaje automático para procesar el audio y entender la señal lingüística subyacente.

Integran información de gramática, sintaxis y contexto de la señal de voz para producir transcripciones precisas. El ASR tiene múltiples aplicaciones prácticas: asistentes de voz (como Siri o Alexa), transcripción de entrevistas o conferencias, subtítulos automáticos en medios audiovisuales, dictado por voz en aplicaciones ofimáticas, y muchos otros. La mayoría de los sistemas ASR actuales se basan en redes neuronales profundas. Se entrena un modelo (por ejemplo, un transformador encoder-decodificador) con un gran corpus de pares audio-texto.

Para procesar un audio, primero se convierte la señal en un espectrograma (representación visual de frecuencias), que se alimenta al modelo. El modelo entonces genera la secuencia de texto correspondiente, a menudo de extremo a extremo, aprendiendo tanto los patrones acústicos como las reglas lingüísticas. Gracias a datos de entrenamiento masivos, estos modelos pueden reconocer palabras en condiciones de ruido moderado o con distintos acentos, superando en muchos casos a los métodos clásicos basados en modelos estadísticos (HMM) utilizados anteriormente.

Modelo Whisper de OpenAI. Un ejemplo destacado de modelo ASR avanzado es Whisper, desarrollado por OpenAI. [7] Según sus creadores, Whisper fue entrenado con 680.000 horas de datos de audio multilingüe supervisados. El uso de un conjunto de datos tan grande y variado hace que Whisper sea muy robusto: maneja bien distintos acentos, ruidos de fondo y lenguaje técnico. Además, Whisper es capaz de transcribir audio en varios idiomas y de traducir estos idiomas al inglés de manera integrada.

La arquitectura de Whisper es de tipo transformer codificador-decodificador de extremo a extremo. El audio de entrada se divide en fragmentos (por ejemplo, de 30 segundos) que se convierten en espectrogramas log-Mel. Un codificador procesa este espectrograma y un decodificador genera la transcripción palabra a palabra. Whisper utiliza tokens especiales para tareas adicionales (por ejemplo, identificar el idioma o incluir marcas de tiempo a nivel de frase). Para adaptarse a diferentes necesidades, Whisper se proporciona en varias variantes de tamaño. Por ejemplo, la variante tiny contiene aproximadamente 39 millones de parámetros,

mientras que la variante large supera los 1.550 millones. Esto permite escoger un modelo más rápido o más preciso según el caso: los modelos pequeños (lightweight) ofrecen inferencia rápida en equipos modestos, y los grandes (large) ofrecen mayor exactitud con mayor coste computacional.

Diarización de locutores y WhisperX. Cuando se procesa audio con múltiples hablantes, es útil además de la transcripción saber quién habla cuándo. La diarización de locutores es el proceso de segmentar un flujo de audio en segmentos homogéneos correspondientes a cada hablante, sin información previa sobre su identidad.

En la práctica, el sistema etiqueta cada parte del audio con un identificador de locutor (por ejemplo, “Locutor 1”, “Locutor 2”) de modo que la transcripción resultante indica qué persona dice cada fragmento. Whisper por sí solo no implementa diarización, genera una transcripción continua del audio. WhisperX es una extensión de Whisper que añade capacidades de diarización y marcas de tiempo más precisas. WhisperX realiza transcripciones rápidas e incluye marcas de tiempo a nivel de palabra. [8]

Además, emplea un pipeline de diarización (basado en la librería Pyannote-Audio) para asignar segmentos de la transcripción al hablante correspondiente.

En otras palabras, WhisperX combina la potencia de Whisper para generar texto con modelos específicos de diarización para etiquetar quién dice cada cosa. De este modo se obtiene una salida organizada: cada palabra transcrita viene acompañada de su intervalo de tiempo exacto y del identificador del hablante. Esto es esencial en aplicaciones como reuniones o entrevistas, donde es necesario distinguir y atribuir el contenido de voz a cada participante.

Marcas de tiempo a nivel de palabra: WhisperX alinea cada palabra con su posición exacta en el audio (gracias a modelos de alineación fonética).

Etiquetas de hablante (diarización): WhisperX segmenta el audio y etiqueta cada segmento con el hablante detectado (p.ej. “Hablaante A: ...”), usando algoritmos de agrupamiento basados en características de voz.

En conjunto, estos conceptos sientan la base fundamental para el resto del trabajo. El lector dispone ya de una comprensión sólida de qué es la IA, cómo funcionan las redes neuronales y sus fases de entrenamiento e inferencia, las diferencias entre hardware y tamaños de modelo, y los principios básicos del reconocimiento de voz avanzado (incluyendo Whisper y WhisperX). Estos fundamentos técnicos serán los pilares sobre los cuales se desarrollarán los temas posteriores del trabajo.

1.1.2. Objetivos del TFG

El objetivo general de este TFG es la creación de una herramienta que ayude a los trabajadores de ADIMA en su trabajo. Esta herramienta toma la forma de una aplicación web para automatizar la transcripción de las entrevistas realizadas.

Para cumplir el objetivo general, nos marcamos los siguientes objetivos específicos:

1. Estudio del modelo de lenguaje a utilizar: Realizamos un pequeño estudio para ver la viabilidad de la herramienta utilizando distintos modelos y sus distintos tamaños. Además hicimos un estudio del hardware necesario para poder ejecutarlos. Estos estudios nos ayudaron a tomar las decisiones adecuadas para asegurar un rendimiento y precisión adecuados.
2. Desarrollo de interfaz: Se desarrollo una interfaz que fuese simple y amigable para asegurar el fácil uso de la aplicación y evitar así problemas a la hora de que los trabajadores la asimilaran en su rutina de trabajo.
3. Ayuda a la gestión del trabajo: Se implementaron algunas funcionalidades para la fácil exportación de los archivos. Además, se introdujeron marcadores para asegurar que párrafos había revisado el trabajador hasta completar la revisión de la transcripción, pudiendo dejar así el trabajo a medias sin necesidad de preocuparse de controlar por donde iban. Por último, se añadió también un sistema de carpetas virtuales para facilitar la ordenación de los archivos.

Desde la perspectiva de la propia organización ADIMA, se han identificado una serie de objetivos fundamentales que esta herramienta debía cubrir para resultar útil en su trabajo:

1. **Asegurar la confidencialidad del contenido:** La aplicación debía funcionar completamente en local, evitando el uso de servidores externos o servicios en la nube, debido a la sensibilidad de la información recogida en las entrevistas.
2. **Facilitar la revisión de contenido:** Los trabajadores necesitaban una forma visual y organizada de revisar las transcripciones, idealmente con indicadores claros de qué partes del texto ya han sido verificadas.
3. **Evitar la pérdida de tiempo en tareas repetitivas:** Automatizar la transcripción permite dedicar más tiempo a tareas humanas esenciales, y menos a la mecanografía o transcripción manual.
4. **Permitir una organización eficaz de los archivos:** Era importante que los documentos pudieran organizarse de manera intuitiva mediante carpetas o categorías, respetando el flujo de trabajo habitual de la organización.
5. **Exportar fácilmente los resultados:** Los trabajadores debían poder exportar las transcripciones a formatos ampliamente utilizados (como .txt o .docx) para su archivo, impresión o envío.
6. **Reducir la barrera tecnológica:** La aplicación debía diseñarse con una interfaz clara y accesible, que pudieran utilizar sin formación técnica avanzada, asegurando una integración fluida en su actividad diaria.
7. **Uso concurrente de la aplicación:** La aplicación debería poder tratar con el tráfico concurrente de todos los trabajadores subiendo archivos, sin que estos se tengan que preocupar por no saturar el servidor.

1.1.3. Alcance del TFG

Este proyecto trata los estudios previos sobre varios modelos de lenguaje para asegurar la elección correcta a la hora de hacer la implementación. También trata la creación y mejora de la interfaz de usuario para la página web, ayudando a la fácil comprensión de la herramienta y ayudando con la gestión de contenido.

Este trabajo, sin embargo, no entrará a revisar otros modelos de lenguaje, quedándose solamente con la opción de software libre de Whisper. Esta elección se hace debido a que la empresa que lo desarrolla, OpenAI, es una de las más importantes en el sector de la inteligencia artificial actualmente y ofrece una gran facilidad para su integración. Tampoco se entrará en este proyecto en el fine-tuning del modelo. Esto es debido principalmente a que estaríamos invirtiendo una gran cantidad de tiempo en un modelo que no sabemos con certeza que será el definitivo. Hasta que los trabajadores de la ONG no valoren cuál de los modelos dados es el más adecuado en cuanto a porcentaje de acierto respecto al rendimiento no se elegirá un modelo final. Teniendo en cuenta también que la opción de adquirir hardware más potente está siendo valorada por la ONG y dicha adquisición abriría paso al posible uso de modelos de mayor tamaño.

1.1.4. Grado de dificultad

La evolución exponencial en todos los campos de la Inteligencia Artificial ha provocado un aumento en su popularidad general pocas veces visto en otro campo de la informática. Es esta popularidad la que ha empujado al aumento en la demanda de soluciones impulsadas por IA a escalas cada vez más pequeñas. Es por esto que a veces afrontar la integración de inteligencias artificiales en soluciones personalizadas a pequeña escala puede llegar a ser una tarea abrumadora y extremadamente confusa. La enorme cantidad de posibilidades disponibles para el usuario y, en muchas ocasiones, la falta de claridad sobre los aspectos importantes de dichas tecnologías hacen que tomar una decisión consciente de las ventajas y desventajas de cada solución sea un trabajo tedioso que el usuario promedio tiende a rechazar.

A esta complejidad se suma el hecho de que muchos de los modelos de inteligencia artificial más avanzados, como los destinados al reconocimiento de voz, requieren no solo conocimientos técnicos para su implementación, sino también una comprensión profunda de sus limitaciones, requisitos de infraestructura y particularidades de configuración. En el caso específico de este proyecto, seleccionar y adaptar un modelo como WhisperX implicó un proceso de aprendizaje continuo, experimentación y resolución de problemas. Esta dificultad refuerza la importancia de realizar un estudio previo y de adoptar una estrategia de implementación que garantice que la solución final no solo sea viable técnicamente, sino también sostenible y adaptada a las necesidades reales de la ONG.

1.1.5. Grado de cumplimiento de los objetivos planteados

En este trabajo se han cumplido casi todos los objetivos propuestos inicialmente, principalmente los necesarios para la mejora de eficiencia en el flujo de trabajo de los trabajadores de la ONG. A continuación, se listan los objetivos cumplidos:

- **Transcripción automatizada:** El objetivo principal del trabajo era agilizar la parte que más tiempo consumía a los trabajadores, la transcripción. El estudio previo sobre los modelos de lenguaje a utilizar y las pruebas realizadas sobre distinto hardware aseguraron un buen equilibrio entre velocidad y precisión en función de las limitaciones de equipamiento que tenía la ONG. Además de asegurar el tratamiento seguro de los datos.
- **Interfaz sencilla:** Como parte secundaria del trabajo, nos enfocamos en facilitar la integración de la herramienta en el flujo de los trabajadores. Para estos utilizamos un framework de Javascript, en este caso Vue.js. Esto se debe a que permite el desarrollo de webs SPA (single page application) que hacen la experiencia final del usuario mucho mas fluida y mas similar al uso de una aplicación que al uso de una web. Además, utilizamos como libreria de estilos Vuetify, debido a que sigue los estándares de material design desarrollados por Google para tener una interfaz de usuario clara y minimalista.

Por desgracia, la falta de experiencia provocaron retrasos a lo largo del proyecto que propiciaron que algunos objetivos menores no se pudiesen cumplir. Esto se debe en parte a que las pruebas con los modelos de ASR fueron muy lentas, atrasando con ello el resto del desarrollo. Principalmente estos retrasos se deben a la lentitud de hacer pruebas de inferencia reales con modelos con los que no se ha trabajado nunca y el intento de optimizarlos para nuestro caso (que devuelvan confianza o el texto en un formato adecuado, por ejemplo).

También se puede sumar a las causas de estos retrasos los problemas en el desarrollo en el frontend y en el backend. Los retrasos en estas partes de la aplicación surgieron principalmente por la falta de experiencia en el desarrollo de aplicaciones full stack, sumándose además a las complejidades que surgen de hacer un sistema con requerimientos complejos como el que tenemos.

A continuación listamos los puntos que no se cumplieron:

- **Despliegue dockerizado de la aplicación:** Inicialmente se contempló que la aplicación estuviera desplegada en un entorno dockerizado, lo cual facilitaría su instalación, mantenimiento y escalabilidad en distintos entornos. Sin embargo, debido a limitaciones en tiempo y recursos, este objetivo no se ha alcanzado en la presente etapa del proyecto. El despliegue en contenedores es una tarea prevista para fases futuras, ya que aportará beneficios significativos en cuanto a portabilidad y replicabilidad del entorno de ejecución.
- **Facilidad en la creación de usuarios:** La creación de usuarios nuevos es actualmente obtusa y compleja, ya que no se ha facilitado ningún menú para ello. Esto se debe principalmente a que el problema de la creación de usuarios y sus permisos para ver y/o editar textos de otras personas pasó desapercibido hasta la reunión final con los trabajadores de la ONG. Además,

los responsables que vieron la aplicación y dieron su visto bueno tampoco tenían claro si debería implantarse un sistema jerárquico que limite la edición de los archivos en función de si el usuario es administrador, es el que ha subido el archivo originalmente o es un tercero.

- **Pruebas con audios reales proporcionados por la ONG:** Aunque la aplicación fue diseñada y probada con audios de prueba simulados, no se ha realizado aún una evaluación exhaustiva con grabaciones reales obtenidas directamente de los trabajadores de la ONG. La ausencia de estas pruebas limita la validación completa del sistema en condiciones reales de uso, lo que es crucial para detectar posibles fallos o mejorar el rendimiento y la precisión en contextos prácticos. Se planea llevar a cabo esta fase de pruebas reales en futuras iteraciones para asegurar la robustez y adecuación del producto a las necesidades específicas de los usuarios.

1.1.6. Potencial aplicación de este trabajo en un entorno académico o industrial

Aplicaciones web como la desarrollada en este proyecto tienen un enorme potencial en todo tipo de entornos. La escalabilidad de la aplicación es prácticamente ilimitada si existe la posibilidad del uso de hardware en la nube, pudiendo utilizar más cantidad de hardware y de mayor calidad. La flexibilidad de una aplicación como esta también es enorme, y se podría utilizar con audios más controlados y de mayor calidad o en entornos donde la calidad sea todavía peor. Todo esto se debe a la gran efectividad que tienen estos modelos de ASR y la posibilidad de mejorarlos para el caso específico de uso a través de fine tuning. Aplicaciones similares a la desarrollada se podrían utilizar en actos oficiales para aligerar la redacción de las actas o en entornos industriales donde la toma de decisiones rápidas y bajo presión pueden acarrear un gran impacto y haga falta una depuración de responsabilidades fiable.

Este potencial no es solo teórico: en el caso concreto de este proyecto, la aplicación va a ser utilizada de forma real y activa por una ONG. Tras una presentación formal del funcionamiento de la plataforma, se mostraron satisfechos con los resultados obtenidos. La posibilidad de subir entrevistas, revisar y editar sus transcripciones y exportarlas para su almacenamiento fue bien valorada, y la experiencia de usuario fue percibida como intuitiva y funcional. El hecho de que el sistema funcione completamente en local, sin necesidad de enviar datos sensibles a la nube, fue especialmente bien recibido dado el carácter confidencial del material que manejan. La validación por parte de los usuarios finales no solo confirma la utilidad real del sistema, sino que además demuestra que este tipo de soluciones pueden implementarse de forma efectiva en contextos sociales donde la tecnología suele llegar con retraso. Esta aceptación por parte de la entidad receptora es, sin duda, uno de los mayores logros del proyecto.

1.2. Estructura de la memoria

Este documento se ha organizado cuidadosamente para guiar al lector a través de todo el proceso desarrollado en el trabajo de fin de grado, desde la motivación inicial y el planteamiento del problema, hasta las conclusiones finales y propuestas de trabajo futuro.

En primer lugar, la sección **Introducción y motivación** contextualiza el TFG, explicando el entorno en el que se enmarca el proyecto, los motivos que justifican su realización y los objetivos concretos que se han planteado alcanzar. Además, se define el alcance del trabajo, el grado de dificultad estimado y el nivel de cumplimiento alcanzado, así como las posibles aplicaciones tanto en entornos académicos como industriales. Esta introducción sirve para establecer las bases conceptuales y prácticas sobre las que se sustenta el resto del documento.

Después, la sección **Gestión del proyecto** detalla el enfoque metodológico adoptado para llevar a cabo el trabajo. Aquí se incluye la planificación temporal, estimación y control de costes, así como un análisis exhaustivo de los riesgos identificados y las estrategias de mitigación implementadas. Esta parte es fundamental para mostrar la viabilidad del proyecto y la capacidad de adaptación ante posibles dificultades, reflejando la madurez en la gestión de proyectos que se ha aplicado.

La tercera sección, **Análisis del problema**, profundiza en la definición precisa del problema que se pretende resolver. En ella se recogen los requisitos de información, funcionales y no funcionales necesarios para una comprensión técnica y práctica del desafío planteado. Además, se establecen las limitaciones y condicionantes que orientan el diseño de la solución, sentando las bases para la fase de desarrollo.

A continuación, la sección **Diseño de la solución** expone la arquitectura global del sistema diseñado para afrontar el problema identificado. Esta parte se divide en subsecciones que detallan los diferentes componentes de la solución, tales como el frontend, el backend, el modelo de reconocimiento automático del habla, la base de datos y la gestión de colas para el procesamiento eficiente. Aquí se justifica la elección tecnológica y se explican las interacciones entre los distintos módulos, destacando la cohesión y escalabilidad del diseño.

La quinta sección, **Implementación**, describe la puesta en práctica del diseño propuesto. Se abordan aspectos como la configuración del entorno de desarrollo, la experimentación y pruebas realizadas con el modelo ASR, el desarrollo de las interfaces de usuario y la implementación de la API REST. También se explican los mecanismos de autenticación y autorización, la gestión de colas para tareas asíncronas, y las funcionalidades de exportación de transcripciones. Finalmente, se presentan posibles extensiones y mejoras futuras, reflejando un enfoque abierto y orientado a la evolución del sistema.

Para cerrar, la sección de **Conclusiones** sintetiza los resultados obtenidos, evaluando el grado de cumplimiento de los objetivos, las aportaciones del trabajo y

las lecciones aprendidas durante el proceso. Se resaltan asimismo las limitaciones detectadas y las recomendaciones para trabajos futuros que permitan ampliar y consolidar los avances realizados.

Complementando el documento, la **Bibliografía** recoge las fuentes consultadas y referenciadas, evidenciando la base teórica que soporta el desarrollo del proyecto.

En resumen, esta estructura facilita un recorrido lógico y didáctico que guía al lector desde los fundamentos hasta la implementación y evaluación final del trabajo, garantizando así una presentación clara y completa del proyecto desarrollado.

2. Gestión del proyecto

2.1. Introducción

La gestión de proyectos es una disciplina fundamental en el desarrollo de cualquier solución informática que aspire a ser aplicada en un entorno real. No basta únicamente con que el producto final funcione correctamente a nivel técnico; es igualmente importante que el proceso que conduce a ese resultado esté bien estructurado, planificado y gestionado de forma eficiente. Esto resulta especialmente relevante en el contexto de un Trabajo de Fin de Grado, donde no solo se evalúa el resultado obtenido, sino también la capacidad del estudiante para organizar los recursos, gestionar el tiempo disponible, anticipar los riesgos y tomar decisiones estratégicas justificadas.

En este capítulo se aborda la gestión integral del proyecto de desarrollo de una aplicación web de transcripción automática para la ONG ADIMA. Se describen los principios metodológicos adoptados, los riesgos previstos y enfrentados, los costes asociados y la planificación temporal estimada, con el objetivo de mostrar que el desarrollo ha seguido una lógica organizada y profesional desde su concepción hasta su implementación.

La metodología empleada ha sido un enfoque ágil y flexible, adaptado a las limitaciones reales del proyecto y al carácter experimental de muchas de sus partes, como es el caso de la integración de modelos de inteligencia artificial. Este enfoque ha permitido realizar iteraciones frecuentes sobre la interfaz, los modelos empleados y la arquitectura del sistema, incorporando progresivamente mejoras en función de los resultados obtenidos y del feedback recibido por parte de los usuarios finales.

También se ha prestado especial atención a la gestión de riesgos. Dado el carácter innovador del proyecto y el uso de tecnologías emergentes o poco conocidas para el desarrollador, se han identificado y clasificado distintos tipos de riesgos con el objetivo de anticipar posibles dificultades y mitigar su impacto. Esta gestión proactiva ha sido fundamental para mantener el proyecto dentro de unos márgenes de control razonables y asegurar su finalización en los plazos establecidos.

Además, se ha realizado un análisis económico de los recursos empleados, distinguiendo entre costes directos e indirectos. Este análisis incluye tanto los gastos derivados del uso del hardware personal como la estimación del coste asociado a una posible implementación en producción mediante un servidor dedicado.

Por último, se presenta la planificación estimada del proyecto, reflejando cómo se ha distribuido el trabajo en las distintas fases del desarrollo: análisis previo, implementación, validación y pruebas. Aunque la planificación original ha sufrido algunos ajustes a lo largo del proceso, se ha mantenido una estructura de trabajo

clara que ha permitido avanzar de forma progresiva hasta alcanzar los objetivos principales.

Esta gestión estructurada es clave para que el producto final no solo funcione, sino que sea también útil, adaptable y reproducible en contextos reales de trabajo como el de la ONG a la que va destinado.

2.2. Planificación estimada

La planificación es una fase crítica en cualquier proyecto de desarrollo software, y cobra especial relevancia en un Trabajo de Fin de Grado, donde los recursos (principalmente tiempo) están claramente acotados y el cumplimiento de plazos forma parte esencial de la evaluación. Desde el inicio del proyecto se elaboró una planificación estimada con el objetivo de distribuir de forma lógica y realista las distintas fases del desarrollo, garantizando la entrega dentro del calendario académico.

La herramienta principal utilizada para la planificación fue un diagrama de Gantt, representado en la Figura, que permitió dividir el trabajo en bloques temporales y visualizar la progresión de las tareas en el tiempo. La planificación fue elaborada con el software Microsoft Project, lo cual facilitó el control de dependencias entre actividades y permitió reordenar tareas cuando surgieron retrasos o imprevistos.

Como se observa en la figura 2.1, el proyecto se estructuró en distintas fases claramente diferenciadas. La primera fase, centrada en la *fase de análisis*, incluyó el estudio de herramientas de reconocimiento automático de voz, el análisis de posibles modelos de inteligencia artificial (con especial atención a Whisper y WhisperX), así como la exploración de soluciones tecnológicas para el backend y frontend de la aplicación, además de las primeras reuniones para definir los requisitos de la aplicación.

Una segunda fase estuvo dedicada a la *fase de diseño* y la preparación del entorno de trabajo. Durante esta etapa se tomaron decisiones clave sobre los lenguajes, frameworks y tecnologías a utilizar: Vue para el frontend, Django y Django REST Framework para el backend, PostgreSQL como base de datos, Celery para la gestión de tareas asíncronas y Redis como sistema de colas. Esta fase también incluyó la definición de los modelos de datos y la elaboración de diagramas de clase y flujos de trabajo.

La fase más extensa fue la de *implementación*, subdividida a su vez en etapas más específicas: desarrollo de la interfaz de usuario, implementación de los endpoints REST, conexión entre frontend y backend, integración del modelo WhisperX, desarrollo del sistema de edición de transcripciones y exportación de documentos.

Por último, se reservó un bloque de tiempo para la *redacción de la memoria*, la generación de documentación técnica y la preparación de la presentación final.

Aunque se intentó respetar la planificación inicial, algunas tareas (como la dockerización del sistema o la implementación completa del sistema de gestión de usuarios) no pudieron completarse en el tiempo previsto, debido a la complejidad técnica y al ajuste fino requerido por otras partes del desarrollo.

En la Figura 2.2 se recoge la hoja de actividades detallada, donde se enumeran las tareas concretas con su correspondiente estimación temporal. Esta hoja fue una herramienta útil para el seguimiento del progreso y para mantener una visión general del estado del proyecto en cada momento.

En conjunto, la planificación estimada fue útil tanto como guía de trabajo como herramienta de control. Aunque la ejecución real presentó ciertas desviaciones respecto a la planificación original, el marco de planificación permitió mantener una ruta clara de avance y ajustar de forma dinámica la distribución de esfuerzos para cumplir con los objetivos principales del TFG.

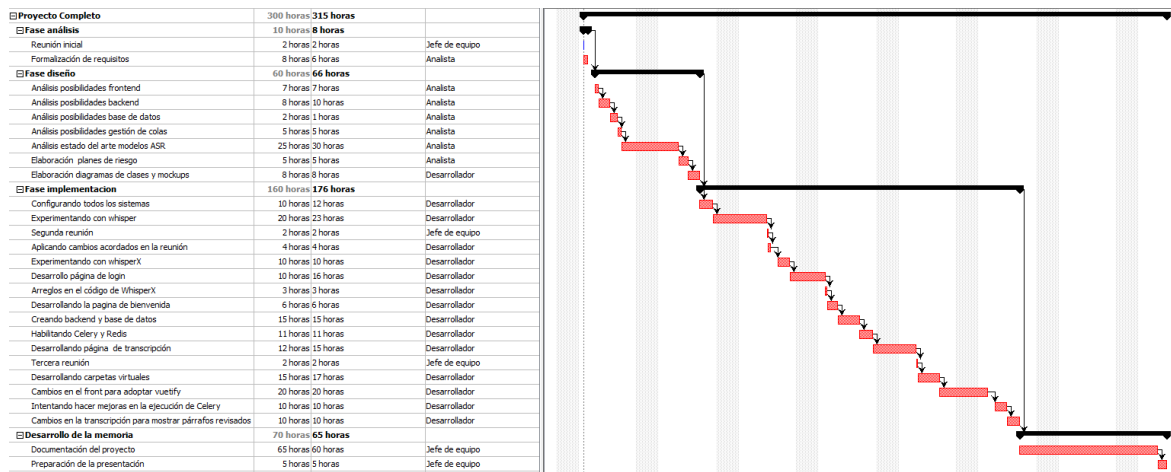


Figura 2.1: Diagrama de Gantt del proyecto

Nombre	Trabajo real
☒ Proyecto Completo	315 horas
☒ Fase análisis	8 horas
Reunión inicial	2 horas
Formalización de requisitos	6 horas
☒ Fase diseño	66 horas
Análisis posibilidades frontend	7 horas
Análisis posibilidades backend	10 horas
Análisis posibilidades base de datos	1 horas
Análisis posibilidades gestión de colas	5 horas
Análisis estado del arte modelos ASR	30 horas
Elaboración planes de riesgo	5 horas
Elaboración diagramas de clases y mockups	8 horas
☒ Fase implementación	176 horas
Configurando todos los sistemas	12 horas
Experimentando con whisper	23 horas
Segunda reunión	2 horas
Aplicando cambios acordados en la reunión	4 horas
Experimentando con whisperX	10 horas
Desarrollo página de login	16 horas
Arreglos en el código de WhisperX	3 horas
Desarrollando la pagina de bienvenida	6 horas
Creando backend y base de datos	15 horas
Habilitando Celery y Redis	11 horas
Desarrollando página de transcripción	15 horas
Tercera reunión	2 horas
Desarrollando carpetas virtuales	17 horas
Cambios en el front para adoptar vuetify	20 horas
Intentando hacer mejoras en la ejecución de Celery	10 horas
Cambios en la transcripción para mostrar párrafos revisados	10 horas
☒ Desarrollo de la memoria	65 horas
Documentación del proyecto	60 horas
Preparación de la presentación	5 horas

Figura 2.2: Actividades del proyecto

2.3. Gestión del coste

Para asegurar un desarrollo completo dentro del presupuesto establecido, es imprescindible una buena gestión de costes. Tanto para evitar retrasos como para asegurar el éxito del proyecto.

Los costes que podemos estimar los dividiremos en directos e indirectos.

2.3.1. Costes directos

Los costes directos se centran principalmente en el personal, e incluyen la retribución de los responsables del proyecto. Esta compensación se calcula en función de las tareas, responsabilidades y funciones específicas que asumen durante el desarrollo de la aplicación.

El objeto de este trabajo requiere el desarrollo de una aplicación web completa y robusta, necesitando por tanto un desarrollador full-stack, un analista y un jefe de proyecto. A continuación tenemos los salarios medios para estos puestos de trabajo.

- Salario desarrollador full-stack: 20 euros/hora
- Salario analista: 25 euros/hora
- Salario jefe de proyecto: 35 euros/hora

El trabajo de fin de grado equivale a 300 horas (12,5 días). Pero la realidad es que este proyecto se ha alargado un poco más, como hemos visto en la hoja de actividades y el diagrama de Gantt. Tomaremos las horas reales trabajadas y las multiplicaremos por el salario de los diferentes componentes del equipo.

$$20\text{euros} \times 180\text{horas} + 25 \times 64\text{horas} + 35 \times 71\text{horas} = 7685\text{euros}$$

2.3.2. Costes indirectos

Los costes indirectos de los materiales en este proyecto incluyen el equipo tecnológico, infraestructura, material informático y licencias. En nuestro caso, al utilizar tecnologías de código abierto, el gasto en licencias será inexistente. También hay que tener en cuenta que en este caso el servidor se desplegará en un ordenador cedido por la facultad. En caso de querer hacer el despliegue en la propia organización para trabajar absolutamente dentro de la red local, haría falta adquirir un ordenador que hiciese de servidor.

A continuación, haremos el desglose del gasto total asociado a la adquisición del equipamiento utilizado a lo largo del proyecto.

Coste amortizado del equipo

Supondremos un portátil de 800 euros que ofrezca un buen balance entre rendimiento y precio. Consideraremos que la vida útil del portátil es de 6 años. En total serían 2191 días. Todo esto daría un gasto de 0,365 euros por día.

El coste amortizado del portátil durante el proyecto sería:

$$0,365\text{euros} \times 12,5\text{días} = 4,56\text{euros}$$

Coste de adquirir un servidor

En caso de querer mover el servidor al edificio para poder actuar siempre dentro de la red local de la ONG, sería necesario adquirir un ordenador. En caso de tomar esta ruta, la inversión en una tarjeta gráfica Nvidia sería lo más recomendable. Esto aceleraría mucho el proceso de inferencia como veremos en las pruebas más adelante. Además, permitiría utilizar modelos de mayor tamaño, aumentando la precisión y reduciendo todavía más la necesidad de corregir la transcripción por parte del trabajador. Un presupuesto que tenga un buen balance entre precio y rendimiento y, además, tenga una tarjeta gráfica capaz de soportar un modelo más grande en su memoria VRAM estaría alrededor de unos 1000 euros en este momento. Este gasto se amortizaría a lo largo de todo el tiempo que dure el ordenador, ya que sería una parte esencial de la organización.

A pesar de esto, asumiremos que el ordenador duraría los 3 años de garantía que tiene. Asumimos también, que trabajaría constantemente y un cuarto del tiempo

al 100 %. Un ordenador como este debería consumir unos 400 vatios en máximo rendimiento y unos 100 vatios cuando no esté transcribiendo. Suponiendo un precio promedio de 0,25 euros/kWh, hacemos los cálculos para ver el gasto diario de tener un servidor así:

$$(90 \text{ W} \times 18 \text{ h} + 350 \text{ W} \times 6 \text{ h}) \times 0,27 \frac{\text{€}}{\text{kWh}} = 0,93 \text{ €/día}$$

Y si amortizamos el ordenador comprado a lo largo de 3 años obtenemos:

$$\frac{1000 \text{ €}}{3 \text{ años} \times 365 \text{ días/año}} = \frac{1000}{1095} \approx 0,91 \text{ €/día}$$

2.4. Gestión de riesgos

Ser conscientes de los riesgos que se corren es imprescindible si queremos un proyecto exitoso. Estos riesgos pueden surgir en cualquiera de las fases del desarrollo, especialmente en proyectos con componentes experimentales y con dependencia de tecnologías emergentes, como es el caso de la inteligencia artificial. Identificar y analizar estos riesgos permite anticiparse a los problemas y establecer planes de contingencia que favorezcan la estabilidad del desarrollo y la calidad del producto final.

La gestión de riesgos es especialmente importante en desarrollos que implican la integración de modelos de IA como WhisperX, ya que depende de bibliotecas sujetas a cambios frecuentes. Además, en el contexto específico de este proyecto, que trata con contenido sensible y exige que todos los datos se procesen localmente, los riesgos relacionados con la seguridad y privacidad cobran todavía más relevancia.

Por ello, en este apartado veremos los tipos de riesgo a los que se enfrenta el proyecto, y se presentará una matriz que permite visualizarlos de forma estructurada y evaluar su impacto. Se sigue un enfoque ágil de desarrollo, lo que permite una planificación más flexible y una respuesta más rápida a riesgos nuevos o cambiantes.

Dada la importancia de la gestión de riesgos en un proyecto como este, en el que se combinan tecnologías experimentales, requisitos técnicos exigentes y usuarios finales no especializados, es fundamental diferenciar los distintos tipos de riesgos que pueden surgir:

Riesgo individual del proyecto: aquellos específicos y únicos asociados a tareas concretas, como por ejemplo la integración de la API de transcripción, la sincronización entre frontend y backend o el procesamiento de audios con ruido o solapamiento de voces.

Riesgo general del proyecto: aquellos que afectan al proyecto en su conjunto, como la falta de tiempo para implementar todas las funcionalidades previstas, la

dependencia de tecnologías externas en evolución (WhisperX), o problemas relacionados con la protección de datos.

Para una correcta gestión de estos riesgos, se han considerado las siguientes acciones clave:

- **Planificar la gestión de riesgos:** se definieron desde el inicio los posibles puntos clave del desarrollo, con especial atención a las dependencias técnicas y la disponibilidad de recursos computacionales.
- **Identificar los riesgos:** se realizó una lista detallada de los posibles riesgos en cada fase del proyecto (análisis, desarrollo, pruebas, entrega).
- **Análisis de riesgos:**
 - **Cualitativo:** priorizando los riesgos según la probabilidad de que ocurran y su impacto sobre la funcionalidad o la entrega del sistema.
 - **Cuantitativo:** evaluando con valores numéricos las combinaciones de probabilidad e impacto, para facilitar la toma de decisiones y definir el orden de mitigación.
- **Planificar respuestas:** se han diseñado medidas preventivas, como pruebas anticipadas con usuarios reales o integración progresiva de componentes críticos.
- **Monitorizar y controlar:** durante el desarrollo se han revisado periódicamente los riesgos identificados, incorporando nuevos si aparecían, y ajustando las respuestas según su evolución.

Cabe destacar que el enfoque seguido en este proyecto es ágil, lo que permite una planificación más flexible y adaptable. Esto ha sido clave para responder con agilidad ante imprevistos técnicos, especialmente en la fase de pruebas.

En cuanto a las estrategias empleadas para afrontar las amenazas detectadas en el proyecto, se han utilizado las siguientes:

- **Aceptar:** en casos de bajo impacto o baja probabilidad, como errores menores de visualización en navegadores concretos.
- **Mitigar:** se han tomado medidas proactivas para reducir la probabilidad o el efecto de riesgos técnicos, como el uso de entornos virtuales y el control de versiones de dependencias críticas.
- **Evitar:** por ejemplo, intentando usar modelos de tamaño reducido, debido al equipamiento del que disponemos para ejecutarlos.
- **Explotar:** se han aprovechado capacidades del modelo WhisperX, como la separación por locutor o mostrar el porcentaje de certeza por frase, como oportunidades para enriquecer las funcionalidades de la aplicación.
- **Mejorar:** en fases tempranas se adaptó el diseño de interfaz, aumentando así la probabilidad de una experiencia de uso satisfactoria.

2.4.1. Riesgo individual del proyecto

Este tipo de riesgo está vinculado a tareas o actividades concretas. En el caso de este proyecto, algunos ejemplos son:

- Fallos en la integración de WhisperX debido a incompatibilidades con las versiones de dependencias.
- Imposibilidad de transcribir correctamente audios con mucho ruido o solapamiento de voces.
- Dificultades técnicas para desplegar la solución de forma local en sistemas sin GPU.
- Problemas con la edición interactiva de la transcripción desde el frontend (Vue).

Estos riesgos afectan tareas específicas, pero pueden retrasar entregas o requerir modificaciones técnicas.

2.4.2. Riesgo general del proyecto

Se trata de riesgos que pueden afectar a múltiples fases o incluso a la totalidad del proyecto. Algunos de ellos son:

- Cambios en la legislación que afecten al tratamiento de datos sensibles, incluso cuando se trabaja en local.
- Obsolescencia rápida del modelo WhisperX o aparición de alternativas con mejor rendimiento.
- Riesgo de que la aplicación no cumpla las expectativas de los usuarios finales, ya sea por problemas de usabilidad o por falta de formación.
- Limitaciones de tiempo y recursos del desarrollador, al ser un Trabajo de Fin de Grado individual.

2.4.3. Matriz de riesgos

Para visualizar los riesgos de forma clara, se construye una matriz que clasifica cada riesgo según su probabilidad de ocurrencia y el impacto que tendría en el proyecto. La escala utilizada es de 1 a 5 para ambos criterios, donde 1 indica baja probabilidad o bajo impacto, y 5 indica muy alta probabilidad o impacto crítico.

ID	Descripción del riesgo	Prob.	Impacto	Nivel (P×I)
R1	Incompatibilidad con dependencias de WhisperX	3	4	12
R2	Fallos de transcripción con audio de baja calidad	4	3	12
R3	Falta de rendimiento sin GPU	2	5	10
R4	Interfaz poco intuitiva para usuarios no técnicos	2	3	6
R5	Falta de tiempo por parte del desarrollador	4	4	16
R6	Cambios normativos en protección de datos	2	3	6

Tabla 2.1: Matriz de evaluación de riesgos

A partir de esta tabla se pueden priorizar las acciones de mitigación. Por ejemplo, el riesgo R5 (falta de tiempo) se abordó estableciendo una planificación realista con márgenes para imprevistos. El riesgo R1 se redujo utilizando entornos virtuales y controlando versiones, además de la realización de pruebas extensas para asegurarnos del correcto funcionamiento antes de comenzar el desarrollo. El riesgo R4 se abordó probando la aplicación con usuarios no técnicos para recoger retroalimentación temprana.

Estrategias adoptadas

Entre las estrategias de gestión de riesgos aplicadas, se destacan las siguientes:

- **Mitigar:** se redujeron riesgos técnicos utilizando herramientas consolidadas y probadas (WhisperX, frontend con Vue.js, etc.).
- **Evitar:** se evitó el uso de servicios en la nube para el procesamiento de datos, eliminando riesgos asociados a la privacidad.
- **Aceptar:** algunos riesgos menores, como retrasos por tareas de documentación, se asumieron sin medidas especiales.
- **Mejorar:** se intentó maximizar el beneficio de los avances técnicos incorporando funciones como identificación de locutores o la posibilidad de visualizar el porcentaje de certeza de una transcripción.

Esta estrategia ha permitido mantener el proyecto bajo control, responder con rapidez a problemas inesperados y garantizar una entrega satisfactoria para el usuario final.

2.5. Metodología

Para afrontar el desarrollo de esta aplicación se ha adoptado una metodología de trabajo flexible, centrada en la iteración continua, el contacto frecuente con los tutores y la validación progresiva de funcionalidades. Dada la naturaleza del proyecto, se optó por un enfoque ágil que permitiese adaptarse a los retos técnicos conforme fueran apareciendo, sin comprometer la viabilidad del sistema.

A diferencia de una metodología en cascada o estrictamente secuencial, donde cada fase debe completarse antes de comenzar la siguiente, en este proyecto se ha trabajado por ciclos iterativos. Cada iteración ha supuesto el desarrollo de una funcionalidad concreta (por ejemplo, la carga de archivos, el procesamiento mediante WhisperX o la edición de la transcripción), su integración en la arquitectura general, pruebas controladas y, cuando ha sido posible, validación con los tutores. Esta forma de trabajo ha permitido detectar errores de diseño en fases tempranas y corregirlos con rapidez, evitando así acumulación de problemas en etapas avanzadas.

Una parte fundamental de esta metodología ha sido el desarrollo incremental del producto mínimo viable (MVP). El objetivo no era construir desde el inicio un sistema completo y complejo, sino alcanzar cuanto antes una versión funcional y usable que permitiera evaluar la validez del planteamiento inicial. A partir de ese núcleo básico, se fueron añadiendo nuevas capas de funcionalidad (como el sistema de carpetas, la exportación de archivos o la marcación de párrafos revisados) sin comprometer la estabilidad del proyecto.

Durante todo el proceso se mantuvo un contacto regular con los tutores, quienes actuaron como usuarios de referencia. Sus observaciones y comentarios fueron esenciales para mejorar la experiencia de uso de la aplicación, adaptar la interfaz al flujo de trabajo de la ONG y garantizar que las funcionalidades desarrolladas se correspondiesen con necesidades concretas.

En lo técnico, se hizo uso sistemático del control de versiones mediante Git para organizar el código y registrar la evolución del proyecto. También se procuró mantener una estructura modular y clara en el backend (mediante Django y Django REST Framework) y en el frontend (mediante Vue), de forma que las distintas partes del sistema pudieran evolucionar de forma independiente y controlada.

Finalmente, en lo relativo a la evaluación de los modelos ASR, se siguió una metodología de experimentación basada en la comparación de resultados con distintas configuraciones. Se analizaron variantes del modelo Whisper y se evaluó el impacto del hardware (CPU vs GPU) en los tiempos de inferencia. Esta parte del trabajo requirió diseñar pequeñas pruebas controladas y extraer métricas cuantitativas (como la tasa de error WER) para poder tomar decisiones justificadas.

En resumen, la metodología seguida ha combinado principios del desarrollo ágil, enfoque iterativo, validación temprana con usuarios reales y análisis técnico riguroso. Esta combinación ha permitido no solo alcanzar los objetivos planteados, sino también hacerlo de una forma que garantiza la mantenibilidad, escalabilidad y

utilidad real del proyecto desarrollado.

2.6. Conclusión

La correcta gestión del proyecto ha sido uno de los pilares fundamentales que han permitido llevar a cabo con éxito el desarrollo de esta aplicación web. A lo largo de este capítulo se ha abordado de forma estructurada la planificación, los recursos, los costes y los riesgos asociados al proyecto, con el objetivo de garantizar un proceso ordenado, eficiente y ajustado a los objetivos definidos desde el inicio.

La planificación estimada, apoyada en herramientas como el diagrama de Gantt y la hoja de actividades, ha permitido organizar el trabajo en fases diferenciadas y asignar tiempos realistas a cada tarea. A pesar de que ciertos elementos (como la dockerización completa del sistema o la gestión avanzada de usuarios) no han podido implementarse por falta de tiempo, el grueso del proyecto se ha completado en los plazos previstos, gracias a una estrategia de trabajo iterativa y flexible.

En cuanto a la gestión económica, se ha realizado una estimación detallada del coste real del proyecto, incluyendo tanto los costes directos (como la dedicación de tiempo del desarrollador) como los costes indirectos (hardware, electricidad, conexión a Internet, etc.). Asimismo, se ha reflexionado sobre el coste potencial de llevar esta solución a producción, contemplando la posible adquisición de un servidor propio o el uso de equipos más potentes para alojar modelos de IA más exigentes. Esta visión económica proporciona una perspectiva realista sobre la viabilidad y sostenibilidad del sistema.

En lo que respecta a la gestión de riesgos, se ha llevado a cabo un análisis exhaustivo que ha permitido anticipar posibles problemas técnicos, organizativos o de planificación. Se han identificado riesgos tanto a nivel individual (relacionados con tareas específicas del proyecto) como a nivel general (que podrían afectar al conjunto del sistema), y se ha elaborado una matriz de riesgos con medidas de mitigación asociadas. Esta labor preventiva ha sido clave para minimizar el impacto de los imprevistos y mantener la continuidad del desarrollo.

También se ha expuesto la metodología de trabajo adoptada, basada en principios ágiles e iterativos. Esta elección ha resultado especialmente adecuada para un proyecto con un componente experimental tan marcado, donde la validación temprana, el ajuste progresivo de decisiones técnicas y el contacto con los usuarios reales de la ONG han tenido un peso fundamental en el éxito del desarrollo.

En conjunto, puede afirmarse que la gestión del proyecto ha sido sólida, bien fundamentada y coherente con los objetivos y recursos disponibles. El enfoque adoptado ha permitido no solo desarrollar una solución técnicamente viable, sino también organizar el trabajo de forma profesional, anticipar dificultades, evaluar los costes asociados y justificar las decisiones tomadas en cada fase. Esta dimensión organizativa refuerza el valor del proyecto más allá de su implementación técnica,

demostrando su viabilidad práctica y su potencial para ser utilizado en un entorno real como el de la ONG Adima.

3. Análisis del problema

3.1. Introducción

En este capítulo se realiza un análisis exhaustivo del problema que se pretende abordar mediante el desarrollo de una aplicación web especializada en el procesamiento y gestión de archivos de audio. El propósito fundamental de esta sección es establecer con claridad los requisitos y características que la aplicación debe poseer para cumplir con su función principal: procesar archivos de audio, obtener de manera automática su transcripción textual y, posteriormente, permitir la edición manual de los resultados obtenidos.

Este análisis preliminar es imprescindible para sentar las bases que guiarán el desarrollo técnico y funcional del sistema, asegurando que la solución final responda adecuadamente a las necesidades y expectativas de los usuarios finales. Por tanto, se realizará una clasificación detallada de los requisitos en diferentes categorías fundamentales: los requisitos de información, que describen qué datos serán manejados y almacenados; los requisitos funcionales, que indican las acciones y procesos que la aplicación debe ser capaz de ejecutar; y finalmente, los requisitos no funcionales, que establecen las condiciones y restricciones bajo las cuales la aplicación debe operar, abarcando aspectos como la seguridad, el rendimiento y la usabilidad.

El análisis que se presenta en este capítulo no solo tiene como objetivo definir qué debe hacer la aplicación, sino también en qué contexto y con qué características, para así garantizar que el producto resultante sea eficiente, confiable y alineado con los objetivos del proyecto y las necesidades del usuario. La correcta identificación y formulación de estos requisitos es una etapa crítica que influirá directamente en las fases posteriores del diseño y el desarrollo de la solución.

3.2. Requisitos de información

Los requisitos de información hacen referencia a los distintos tipos de datos que la aplicación web deberá gestionar, almacenar y manipular durante su funcionamiento. La correcta definición de estos datos es fundamental para garantizar que la aplicación tenga una estructura coherente y funcional que permita el acceso, actualización y eliminación eficiente de la información, así como su posterior recuperación para distintos fines.

En el contexto de la aplicación propuesta, se identifican principalmente tres grandes bloques de información:

- **Usuarios:** Este bloque contiene los datos necesarios para identificar y autenticar a cada usuario que accede a la plataforma. Se incluye el nombre o

identificador único de usuario, así como la contraseña, que debe ser almacenada utilizando métodos de cifrado y hashing seguros para proteger la privacidad y evitar accesos no autorizados. Además, se contemplan posibles datos adicionales relacionados con la gestión del perfil del usuario y sus preferencias dentro de la aplicación.

- **Archivos de audio:** Este conjunto de datos está relacionado con los archivos que los usuarios suben para su procesamiento. Cada archivo de audio debe estar asociado a un nombre descriptivo, el formato en que se encuentra (por ejemplo, MP3, WAV, etc.), la duración total del audio y la fecha en que fue subido al sistema. Adicionalmente, se prevé la implementación de un sistema de etiquetas o metadatos que los usuarios puedan asignar a los archivos, facilitando así la organización, búsqueda y recuperación eficiente de los mismos en el futuro. Esta clasificación por etiquetas resulta fundamental para mejorar la experiencia de usuario y la gestión del contenido almacenado.
- **Transcripciones:** Finalmente, se deben almacenar las transcripciones generadas automáticamente a partir de los archivos de audio procesados. Estas transcripciones consisten en bloques de texto separados en párrafos, donde cada párrafo está asociado a un locutor específico y una marca temporal que permite navegar por la transcripción sincronizadamente con el audio. Además, para ofrecer una experiencia de usuario enriquecida, cada segmento de texto debe incluir un porcentaje de certeza o confianza calculado por el modelo de reconocimiento de voz, lo cual permite evaluar la fiabilidad de cada parte de la transcripción. Es esencial que tanto el texto de los párrafos como la identificación del locutor sean editables por parte del usuario, para corregir posibles errores o ajustar el contenido según las necesidades específicas.

En resumen, estos datos deben ser almacenados en una estructura clara, coherente y flexible que permita operaciones como búsquedas avanzadas, modificaciones y borrados, garantizando así una gestión completa y adaptada a las necesidades de la aplicación y sus usuarios.

3.3. Requisitos funcionales

Los requisitos funcionales constituyen el conjunto de funcionalidades y operaciones que la aplicación debe ser capaz de llevar a cabo para cumplir con su propósito. En esta sección se describen las acciones principales que el sistema debe ofrecer a los usuarios, así como los procesos internos que deben ejecutarse para conseguir una experiencia completa y satisfactoria.

A continuación, se listan y describen en detalle las funcionalidades más importantes previstas en el proyecto:

- **Autenticación de usuarios:** La aplicación debe permitir a los usuarios registrarse y acceder mediante un sistema seguro de autenticación. Esto incluye la validación de credenciales, la gestión de sesiones y la protección de

la información sensible durante la comunicación con el servidor, garantizando que solo usuarios autorizados puedan acceder a sus datos personales y archivos.

- **Subida y eliminación de archivos de audio:** Los usuarios deben poder cargar archivos de audio desde su dispositivo local al servidor, así como eliminar aquellos archivos que ya no deseen conservar. Esta funcionalidad debe ser intuitiva, rápida y segura, soportando diferentes formatos de audio y permitiendo una gestión ordenada de los archivos subidos.
- **Procesamiento automático de los audios mediante WhisperX:** Una vez que un archivo es subido, la aplicación debe ser capaz de invocar el modelo WhisperX para realizar una transcripción automática del contenido de audio. Este proceso debe ser gestionado de forma eficiente y transparente para el usuario, con mecanismos que informen del estado y progreso del procesamiento.
- **Generación de transcripciones diferenciadas por locutor y tiempo:** La transcripción resultante debe segmentarse según el locutor identificado y asociarse con marcas temporales específicas que permitan una navegación sincronizada entre el texto y el audio original. Esto facilita la comprensión y revisión del contenido transcrito.
- **Visualización y edición del texto resultante por parte del usuario:** La aplicación debe mostrar la transcripción de forma clara y organizada, permitiendo que el usuario pueda revisar, modificar y corregir el texto y la identificación de los locutores según sea necesario. Esta capacidad de edición es esencial para mejorar la calidad y precisión de la transcripción final.
- **Reproducción sincronizada del audio con la transcripción:** Para mejorar la experiencia de usuario, el sistema debe ofrecer un reproductor de audio integrado que sincronice la reproducción con la visualización del texto transcrito, destacando la parte correspondiente del texto conforme avanza el audio.
- **Exportación de la transcripción en distintos formatos:** Finalmente, se debe proporcionar la posibilidad de exportar las transcripciones editadas en formatos estándar de texto como .txt, .docx u otros, para facilitar su uso fuera de la aplicación y su integración en otros entornos o herramientas.

Estas funcionalidades constituyen el núcleo operativo de la aplicación y definen la experiencia que los usuarios podrán tener al interactuar con el sistema.

3.4. Requisitos no funcionales

Los requisitos no funcionales describen las propiedades, restricciones y condiciones bajo las cuales la aplicación debe operar, sin definir acciones específicas, pero condicionando el diseño y la implementación para garantizar un

producto de calidad. En este proyecto, se han identificado los siguientes requisitos no funcionales esenciales:

- **Seguridad:** La protección de los datos de usuario es prioritaria. La aplicación debe implementar mecanismos de autenticación robustos que utilicen cifrado para proteger las contraseñas y garantizar la confidencialidad e integridad de la información transmitida entre cliente y servidor. Además, se deben considerar aspectos relacionados con la autorización y protección contra accesos indebidos.
- **Rendimiento:** El sistema debe ser capaz de procesar los archivos de audio y generar la transcripción en un tiempo razonable y aceptable para el usuario. Esto implica optimizaciones tanto en el procesamiento interno como en la gestión de recursos para evitar esperas excesivas o bloqueos.
- **Usabilidad:** La interfaz de usuario debe ser intuitiva, accesible y sencilla de utilizar, incluso para personas sin conocimientos técnicos avanzados. Esto incluye un diseño claro, opciones bien organizadas y una experiencia de navegación fluida que facilite el acceso a todas las funcionalidades.

Estos aspectos no funcionales son vitales para garantizar la calidad global del sistema y la satisfacción del usuario final, condicionando muchas de las decisiones técnicas y de diseño que se adoptarán en etapas posteriores.

3.5. Conclusiones

En conclusión, en este capítulo se ha llevado a cabo una definición clara y estructurada del problema que pretende resolver la aplicación web en desarrollo. A través del análisis detallado de los requisitos de información, funcionales y no funcionales, se ha establecido una base sólida que orientará el diseño técnico y la implementación del sistema.

Se ha identificado el tipo de datos que la aplicación debe manejar, desde la gestión de usuarios y archivos de audio hasta la estructura y características de las transcripciones automáticas, asegurando que sean editables y adecuadamente organizadas. Asimismo, se han enumerado las funcionalidades que el sistema debe ofrecer, desde la autenticación hasta la exportación de resultados, pasando por la integración de modelos avanzados de reconocimiento de voz y la interacción sincronizada entre audio y texto.

Por último, se han considerado las condiciones y requisitos que aseguran la calidad del producto, como la seguridad, el rendimiento y la usabilidad, aspectos imprescindibles para una solución eficiente y satisfactoria.

Esta definición clara y completa de los requisitos será fundamental para orientar las etapas sucesivas del proyecto, facilitando la planificación, el desarrollo y la evaluación de la aplicación, con el objetivo de lograr un producto final que cumpla con las expectativas y necesidades de los usuarios, garantizando una experiencia óptima y resultados fiables.

4. Diseño de la solución

4.1. Introducción

Una vez definidas las funcionalidades clave que debe cumplir la aplicación y analizados los posibles desafíos técnicos y de experiencia de usuario que pueden surgir durante el desarrollo, es el momento de establecer las decisiones de diseño que marcarán el rumbo de la implementación. Estas decisiones afectan tanto a la arquitectura técnica como a la experiencia de usuario, y serán determinantes para garantizar la usabilidad del sistema final.

En este capítulo se recopilan y desarrollan todas las decisiones importantes que se tomaron antes de iniciar la fase de implementación, así como las razones que las motivaron. Algunas de estas elecciones han sido revisadas y modificadas a lo largo del proyecto conforme se adquiría una mayor comprensión de los requisitos reales y se detectaban limitaciones o ventajas no previstas inicialmente.

Estas decisiones abarcan desde la selección de tecnologías para cada una de las capas de la aplicación (frontend, backend, base de datos, etc.), hasta aspectos visuales, estructurales y de organización lógica del sistema. Se pondrá especial atención en justificar cada elección desde un punto de vista técnico, funcional y práctico, teniendo en cuenta factores como la facilidad de desarrollo, la eficiencia, el rendimiento y la curva de aprendizaje.

A lo largo del capítulo, se irán desgranando las distintas piezas que forman la arquitectura global del proyecto, abordando cada una por separado para facilitar la comprensión del sistema completo. Este análisis incluye tanto comparativas entre distintas alternativas tecnológicas como descripciones detalladas del estilo visual y del modelo de interacción con el usuario.

Este enfoque no solo facilita el desarrollo posterior del sistema, al tener claras las líneas maestras a seguir, sino que también permite mantener la coherencia y la calidad del producto a medida que se implementan nuevas funcionalidades o se realizan modificaciones.

4.2. Arquitectura global

La aplicación web tendrá varios componentes que la constituirán. Las partes relevantes sobre las que tendremos que tomar decisiones de peso son:

- Frontend: La parte frontal de la aplicación determinará casi por completo la experiencia de usuario. Debemos centrarnos en hacer una interfaz sencilla y en la que sea fácil identificar la situación de los audios y transcripciones.

- **Backend:** La parte trasera de la aplicación será la columna vertebral de la aplicación. Encargada de gestionar todos los procesos que necesitamos para el correcto funcionamiento de la aplicación. Buscaremos una tecnología que sea rápida y fácil de implementar.
- **Modelo:** El modelo de IA que elijamos determinará la otra gran parte de la experiencia. Un modelo adecuado hará que la implementación del código para usarlo sea mas sencillo. Por último, un tamaño correcto de dicho modelo hará que las transcripciones se hagan lo suficientemente rápido y con un margen aceptable de error.
- **Base de datos:** La base de datos guardará toda la información que requiere la aplicación para funcionar. Esta elección será menos relevante, ya que casi cualquier software de base de datos moderno nos ofrece lo que necesitamos.
- **Gestión de colas:** La aplicación debe soportar el acceso concurrente de varios usuario, con todo lo que ello conlleva. La tarea más difícil será la gestión de las colas cuando se tengan que transcribir varios archivos a la vez.

Teniendo claro los componentes más relevantes de la aplicación, ahora los iremos revisando uno a uno. Tomaremos las decisiones necesarias para asegurar un buen desarrollo y funcionamiento, y las razonaremos. Estas decisiones variarán en importancia, e irán desde elecciones de software con el que desarrollaremos hasta elecciones de estilo.

4.2.1. Frontend

El frontend es una de las piezas clave de cualquier aplicación web, ya que es el punto de contacto directo entre el usuario y el sistema. En este proyecto, tiene especial relevancia, ya que buscamos que el proceso de subir archivos, visualizar transcripciones y editarlas sea fluido, claro y accesible para perfiles no especializados. La interfaz debe guiar al usuario a través de las distintas funcionalidades sin necesidad de instrucciones complejas. Por ello, se ha decidido implementar la aplicación como una **SPA (Single Page Application)**, lo que permite reducir los tiempos de carga, mejorar la fluidez en la navegación y ofrecer una experiencia más dinámica y moderna.

Se requiere una interfaz limpia, intuitiva y funcional, que no distraiga del objetivo principal: procesar y trabajar con grabaciones de audio y sus respectivas transcripciones.

Comparación de frameworks frontend

Para el desarrollo del frontend, se han considerado tres de los frameworks más populares del ecosistema web actual: Angular, React y Vue. A continuación se presentan sus principales características, además de sus ventajas y desventajas:

- **Angular:** desarrollado por Google, es un framework completo y estructurado que incluye todo lo necesario para construir aplicaciones de gran escala.



Figura 4.1: Mockup de pantalla de inicio de sesión

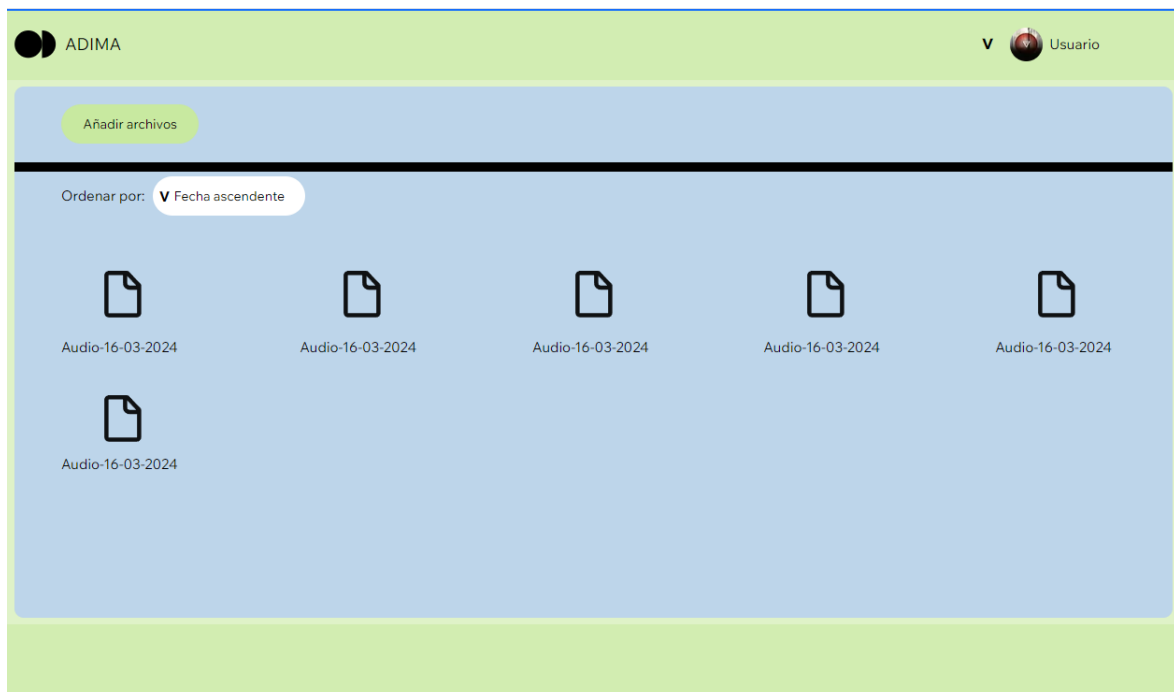


Figura 4.2: Mockup de pantalla de bienvenida



Figura 4.3: Mockup de pantalla de transcripción

Angular se basa en TypeScript, lo que aporta robustez y escalabilidad al código. Cuenta con una arquitectura basada en componentes y ofrece soluciones integradas para el enrutamiento, formularios, pruebas y consumo de APIs. Sin embargo, uno de sus principales inconvenientes es su complejidad inicial: tiene una curva de aprendizaje pronunciada y requiere la adopción de muchas convenciones y estructuras, lo cual puede ralentizar el desarrollo y resulta menos adecuado para un equipo de una sola persona o proyectos de medio alcance como este.

- **React:** desarrollado por Meta, React es una biblioteca enfocada exclusivamente en la creación de interfaces de usuario. [9] Su gran ventaja es la flexibilidad que ofrece al combinarse con librerías externas para tareas específicas como gestión de rutas o estado. Su ecosistema es muy amplio, y está respaldado por una enorme comunidad. React permite escribir componentes reutilizables y tiene un rendimiento excelente. No obstante, esta misma flexibilidad puede convertirse en una desventaja si no se gestiona correctamente: obliga a tomar decisiones sobre múltiples aspectos desde el inicio, lo que añade cierta complejidad a la configuración inicial. Además, aunque su curva de aprendizaje es menos acusada que la de Angular, requiere tiempo para dominar buenas prácticas.
- **Vue:** Vue es un framework progresivo que permite construir aplicaciones web de forma gradual, sin imponer una estructura rígida. [10] Su sintaxis clara y sencilla, junto con una documentación excelente, lo convierten en una opción especialmente interesante para proyectos que requieren agilidad y claridad en el código. Vue ofrece una integración fluida de HTML, CSS y JavaScript, lo que permite a los desarrolladores centrarse en el desarrollo sin necesidad

de aprender un nuevo lenguaje o adaptar sus herramientas. Además, incluye herramientas oficiales como Vue Router para la navegación y Pinia para la gestión del estado global.

En este proyecto, se ha optado por usar Vue debido a su ligereza, rendimiento y al conocimiento previo que tenía el desarrollador del framework. Esta elección nos permitirá acelerar el desarrollo del frontend sin hacer sacrificios en el apartado de calidad.

Aplicaciones SPA

La aplicación será una SPA (Single Page Application), es decir, una aplicación web que carga una sola página HTML y que actualiza dinámicamente el contenido conforme el usuario interactúa con la interfaz. Esto produce una mejora considerable de la experiencia del usuario, ya que no se producen recargas completas de página, y las transiciones entre vistas son inmediatas. También permite un manejo más eficiente del estado de la aplicación, lo cual es crucial para características como la edición de transcripciones en tiempo real, la reproducción de audios o la visualización de archivos.

Al utilizar Vue como base del desarrollo, esta arquitectura será fácilmente implementable mediante Vue Router para la navegación interna y componentes dinámicos para gestionar vistas como “página de navegación”, o “edición de transcripciones”.

Decisiones de diseño visual

El diseño visual del frontend no solo cumple una función estética, sino que también tiene un impacto directo en la usabilidad, accesibilidad y claridad de la aplicación. Por ello, se ha decidido tomar como base la identidad visual de la página web de la ONG de Adima, de forma que el estilo general de la aplicación sea coherente y familiar para los usuarios habituales de la organización. Este enfoque refuerza la sensación de integración y facilita la adopción de la herramienta en contextos profesionales o institucionales.

- **Colores:** se utilizará una paleta de colores centrada en el color corporativo de la ONG (azul marino principalmente), combinada con colores neutros como grises suaves y blanco para los fondos. Se buscará mantener un contraste adecuado entre el fondo y el texto para asegurar la legibilidad. Los mensajes de alerta o errores usarán colores tradicionales como rojo o amarillo, mientras que los mensajes de éxito usarán verdes intensos. Además, en las transcripciones el fondo de los párrafos cambiará dependiendo del locutor que hable. También cambiará el color del texto en función de si la confianza del modelo en ese párrafo es buena o no. Tenemos que señalar también el cambio de color del texto a negro cuando el usuario pulsa enter en un párrafo, asegurando así al usuario de una forma muy visual que párrafos ha revisado y cuáles no. Por último, para indicar de manera correcta que párrafo está reproduciendo el audio en todo momento, este se moverá al centro de la pantalla y aparecerá una marca verde para que el usuario lo pueda identificar.

- **Botones:** los botones principales, como los de “Guardar transcripción” o “Exportar documento”, tendrán un diseño prominente, bordes redondeados y serán fáciles de identificar. Todos los botones tendrán animaciones sutiles al pasar el cursor, para dar una sensación de dinamismo sin sobrecargar visualmente la interfaz.
- **Listado de archivos:** para facilitar la navegación a través de los archivos se crearon dos sistemas, uno de navegación por carpetas y otro de etiquetas. El primero permite crear carpetas virtuales para poder gestionar de manera jerárquica los archivos subidos, asegurando correcto orden en la aplicación en el futuro. Las etiquetas de cada archivo son creadas por el usuario y puede realizarse una búsqueda a través de los archivos con esas etiquetas rellenando un input de texto.

En conjunto, estas decisiones de diseño están orientadas a ofrecer una experiencia de usuario coherente, intuitiva y agradable, alineada con el estilo visual de la web de la ONG y ajustada a los requisitos funcionales del sistema.

4.2.2. Backend

El backend constituye el núcleo lógico de la aplicación. Es la parte encargada de procesar peticiones, gestionar datos, autenticar a los usuarios y coordinar la comunicación entre los distintos apartados de la aplicación (modelo de IA, base de datos, frontend, etc.). Su correcto desarrollo es esencial para garantizar el correcto funcionamiento y seguridad de la aplicación.

En este proyecto, se ha optado por desarrollar un backend que proporcione una API REST moderna, robusta y bien estructurada, capaz de dar servicio a una aplicación SPA como la descrita anteriormente. Para ello, se han considerado varias tecnologías ampliamente utilizadas en la industria: **Spring Boot (Java)**, **FastAPI (Python)** y **Django (Python)**. A continuación, se comparan sus principales características y su idoneidad en el contexto de esta aplicación.

Comparación de tecnologías para el backend

- **Spring Boot (Java):** es uno de los frameworks más robustos y ampliamente adoptados en entornos empresariales. [11] Ofrece un conjunto muy completo de herramientas para crear APIs REST, manejar bases de datos, seguridad, etc. Su ecosistema cuenta con soporte profesional y empresarial. No obstante, su uso implica una mayor complejidad inicial: el entorno de desarrollo, la configuración de dependencias y la propia sintaxis del lenguaje Java hacen que el tiempo de desarrollo se alargue considerablemente en comparación con frameworks más ligeros. Para un proyecto ágil y con un equipo reducido, Spring puede resultar excesivo.
- **FastAPI (Python):** es un framework moderno, rápido y ligero, ideal para construir APIs REST de forma eficiente. [12] Ofrece validación automática de datos mediante Pydantic, documentación interactiva integrada con OpenAPI,

y un rendimiento excelente gracias a su compatibilidad con ASGI y su diseño asíncrono. Sin embargo, su ecosistema aún está en crecimiento, y aunque permite una gran libertad, muchas tareas (como autenticación compleja, manejo de permisos o gestión de usuarios) requieren configuraciones personalizadas que consumen tiempo y pueden introducir errores si no se manejan con cuidado.

- **Django (Python):** es un framework web de alto nivel que promueve el desarrollo rápido y limpio. [13] Django destaca por su robustez, seguridad por defecto, gran comunidad y abundante documentación. Pero su mayor ventaja en este proyecto es la integración con **Django REST Framework (DRF)**, un conjunto de herramientas potente y flexible para construir APIs REST de manera profesional. DRF permite definir endpoints con precisión, establecer permisos detallados, validar automáticamente los datos y trabajar cómodamente con formatos como JSON. Además, Django incluye un sistema de autenticación, middleware configurable, integración con ORM y herramientas de administración que reducen significativamente el tiempo de desarrollo.

Gracias a todo esto, se ha elegido **Django como tecnología para el backend**, ya que proporciona una solución completa, ideal para desarrollar una API segura y consistente que sirva de puente entre el frontend y el procesamiento de los audios.

API REST y Django REST Framework

La manera principal de comunicación entre el frontend y el backend será una **API RESTful**, desarrollada con Django REST Framework. Esta API permitirá a los usuarios:

- Subir y eliminar archivos de audio.
- Consultar y editar transcripciones procesadas.
- Descargar resultados en distintos formatos.
- Autenticarse mediante credenciales seguras.
- Obtener el estado de las transcripciones en curso.

El uso de DRF nos permitirá definir de forma clara los recursos y establecer rutas adecuadas. Toda la comunicación se realizará en formato **JSON**, asegurando compatibilidad total con el frontend. Además, DRF nos permitirá generar las rutas de la API de manera estructurada, permitiendo que el código sea organizado y fácil de entender.

Seguridad en las peticiones

Uno de los aspectos fundamentales en cualquier sistema web moderno es la **seguridad de las comunicaciones** y el control de acceso a los recursos. Como hemos recalcado en otras ocasiones, este aspecto es especialmente importante en

nuestro desarrollo. Django y DRF ofrecen soporte nativo para muchos mecanismos de seguridad, los cuales se aplicarán en este proyecto:

- **JWT (JSON Web Tokens):** se utilizará un sistema de autenticación basado en tokens JWT para gestionar sesiones de usuario de forma segura. Cada vez que un usuario se autentique, se le entregará un token que incluirá su identidad y sus permisos. Este token será enviado en cada petición al backend, y validado para permitir o denegar el acceso a recursos.
- **CORS (Cross-Origin Resource Sharing):** dado que el frontend y el backend se desarrollarán y desplegarán en dominios distintos durante el desarrollo, será necesario configurar correctamente el middleware de CORS en Django. Esto permitirá que el navegador acepte peticiones cross-origin únicamente desde dominios autorizados, evitando vulnerabilidades de acceso entre sitios.

La elección de Django como tecnología para el backend garantiza un entorno de desarrollo seguro, bien estructurado y con una comunidad muy activa. Gracias a su integración con Django REST Framework, se facilita la creación de una API REST potente y mantenible, que cubre todas las necesidades funcionales de la aplicación: desde la gestión de usuarios y archivos hasta la comunicación con los modelos de transcripción y la entrega de resultados al frontend.

La combinación de robustez, documentación, madurez del ecosistema y herramientas integradas convierte a Django en una solución ideal para este tipo de aplicaciones orientadas a la web, facilitando mucho el trabajo del desarrollador y acelerando el proceso de desarrollo.

4.2.3. Modelo

En un proyecto como el que vamos a llevar a cabo, es esencial elegir un modelo ASR que ofrezca alta precisión y fácil integración. Ahora haremos una comparación entre algunas de las opciones que se barajaron como modelo de ASR a integrar:

- **Whisper:** Whisper es un modelo ASR multilenguaje entrenado en 680,000 horas de datos diversos. Entre sus fortalezas figura una notable robustez ante acentos, ruido de fondo y terminología técnica. Whisper además, ofrece distintos tamaños desde tiny hasta large. Estos tamaños cambian velocidad de inferencia por precisión, y encontrar el tamaño que menos sacrificios haga será una tarea de experimentación que hagamos más adelante. Este es uno de los modelos que más precisión ofrecen, siempre dependiendo del tamaño que se elija. La principal ventaja es que tiene una gran comunidad a su alrededor y gran cantidad de contenido educativo sobre él, que si lo añadimos a una fácil integración, nos da como resultado una muy buena opción para este proyecto.
- **WhisperX:** WhisperX es una extensión de Whisper desarrollada para mejorar las marcas de tiempo y agregar diarización de hablantes. Refina la salida de Whisper usando alineamiento forzado y obtiene marcas de tiempo precisas a nivel de palabra y oraciones. Promete un mejor rendimiento y una mayor

precisión que Whisper con el único coste de tener que desarrollar algo más de código, aumentando ligeramente su complejidad.

- **Vosk:** Vosk es un conjunto de herramientas ASR enfocado en la eficiencia. [14] Soporta múltiples idiomas y funciona incluso en dispositivos de baja potencia. Tiene modelos extremadamente pequeños y también es fácil de instalar. Aunque su enfoque principal en dispositivos sin GPUs puede sonar razonable para nuestro caso, este requiere algunos conocimientos para ajustar los modelos. Además, a pesar de su eficiencia, en entornos un poco más complejos como puede ser una entrevista entre dos personas que se ha grabado desde un móvil, baja considerablemente su precisión.
- **DeepSpeech:** DeepSpeech fue desarrollado por Mozilla, está optimizado para correr en tiempo real incluso en equipos poco potentes. [15] Su integración es un poco más tediosa ya que se deben descargar manualmente archivos de modelo y vocabulario. La principal desventaja de un modelo tan ligero es que en entornos algo más complejos pierde en precisión respecto a Whisper.

Elección de WhisperX

Para una aplicación de transcripción de entrevistas en español concluimos que WhisperX ofrece el mejor equilibrio. Garantiza transcripciones de calidad incluso en entornos con ruido moderado ya que hereda la precisión de Whisper. A su vez, incorpora marcas de tiempo muy precisas y una diarización extremadamente eficaz. Todo esto lo veremos más adelante con las pruebas que se le realizaron a los dos modelos.

4.2.4. Base de datos

En el entorno actual de bases de datos hay dos paradigmas que podríamos seguir: relacionales (SQL) o no relacionales (NoSQL). [16] En nuestro caso se eligió una base de datos relacional, más concretamente PostgreSQL.

En la siguiente imagen podemos ver el primer diagrama de clases que se diseñó. Las clases principales serían el usuario, el archivo, la transcripción y los tiempos_textos. Este diagrama no representa la realidad final de la aplicación ya que sufrió cambios a lo largo del desarrollo. El cambio más grande fue la adición de la clase carpeta, necesaria para poder organizar los archivos de manera clara en carpetas virtuales en el navegador. El diagrama final se verá más adelante.

Además del conocimiento previo del software que tenía la persona desarrollando el proyecto, se eligió también por otros motivos:

- **Consultas complejas:** SQL permite consultas complejas que pueden resultar útiles en el backend.
- **Claridad:** el modelo de datos se diseña explícitamente. Esto permite que las relaciones y entidades se puedan ver claramente desde el principio, lo que ayuda con el seguimiento de las actualizaciones en el modelo. Gracias a la

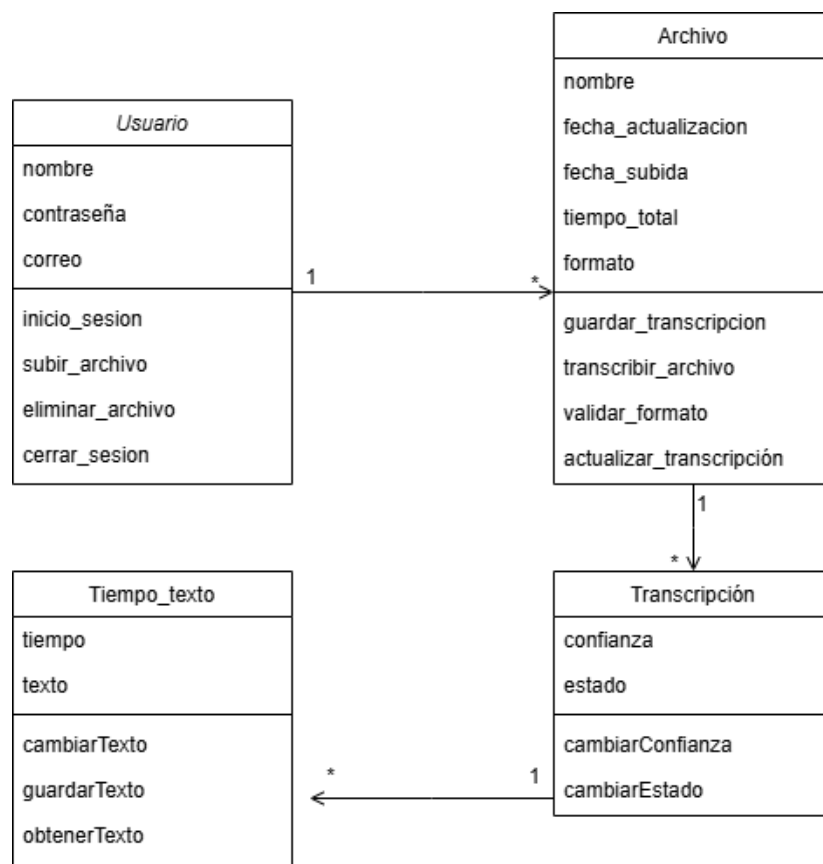


Figura 4.4: Primer diseño UML de la base de datos

integración de Django con PostgreSQL también podemos hacer migraciones con un solo comando que se ven reflejadas en la base de datos automáticamente. Si juntamos eso con la capacidad de la herramienta de gestión de PostgreSQL, PGAdmin, para crear diagramas entidad-relación, obtenemos como resultado un flujo rápido de actualización del modelo de base de datos en caso de que detectemos errores y hagan falta cambios.

Es por esto que PostgreSQL fue la única opción considerada para gestionar la base de datos, ayudando con su buena integración con django a que se produjeran menos errores de programación y acelerando así el desarrollo.

4.2.5. Gestión de colas

En esta aplicación web varios trabajadores de la ONG accederán a la web de manera simultánea. Además, estos usuarios podrían subir sus archivos para que comiencen a transcribirse de manera concurrente. Por esto, es necesario gestionar las colas de las tareas.

Para nuestro proyecto solo se valoró Celery como librería de gestión. [17] Celery es un sistema de gestión de colas asíncrono que será el encargado de ejecutar el modelo whisperX sobre los audios que suba el usuario.

Redis actúa en este proyecto como broker de mensajes, es decir, como intermediario entre la aplicación principal y los trabajadores de Celery. [18] Cuando un usuario sube un archivo de audio, se crea una tarea asíncrona que se envía a Redis, y queda en cola hasta que el worker esté disponible y la recoja para ejecutarla. Lo que conseguimos con esto es que la transcripción quede desacoplada del hilo principal del servidor web, evitando así que la aplicación se bloquee cada vez que se sube un archivo.

El flujo entonces quedaría de la siguiente manera:

- **Subida del archivo**
- **El backend guarda el archivo y lanza la tarea**
- **Redis pone en cola la tarea**
- **El trabajador de celery toma la tarea y genera la transcripción**
- **El resultado se guarda en base de datos y queda visible desde el front end**

La combinación de Celery y Redis es una solución sólida, eficiente y escalable para manejar una tarea tan intensiva como la transcripción de audios. Permite que el front end se mantenga responsivo y se ajusta a las necesidades prácticas de la ONG

5. Implementación

5.1. Introducción

En este capítulo explicaremos todo el proceso de implementación y desarrollo del proyecto. Veremos los pasos que se siguieron y cómo actuamos en función de las decisiones tomadas en el apartado de diseño. En algunas ocasiones fue necesario tomar decisiones que eran distintas a las tomadas en la fase de diseño, esto también lo veremos a lo largo del capítulo.

5.2. Configuración del entorno de desarrollo

Para este proyecto podemos identificar tres partes diferenciadas que tendremos que desarrollar: el frontend, el backend y el script para hacer la transcripción. En el desarrollo de los tres usamos VSCode como entorno de desarrollo. [19] Para gestionar las versiones se usó Git. Para facilitar el control de las versiones del proyecto se gestionaron las tres partes a la vez a través de una única rama de GitHub. [20]

En las partes del desarrollo de python se instalaron entornos virtuales, esto nos ayudó a la hora de aclarar las dependencias y tenerlas registradas, ya que podemos escribir todas las dependencias en un archivo de texto con un solo comando. En el apartado del frontend no hizo falta hacer esto, ya que el propio paquete con el que levantamos la primera aplicación de Vue nos ofrece un apartado con todas las dependencias actuales.

También fue necesario instalar varias herramientas auxiliares para poder completar el desarrollo. Además de los ya mencionados VSCode y Git, se instaló también Docker, que nos sirvió para crear la imagen de Redis que necesita Celery para conectarse. Por último, se instaló Postman para poder hacer pruebas más específicas en los endpoints desplegados de la api creada.

5.3. Pruebas del modelo ASR

Antes de comenzar el desarrollo de la aplicación, debemos hacer un estudio de qué modelos de ASR podemos utilizar. El modelo Whisper de OpenAI es uno de los más utilizados debido a su buen rendimiento en la transcripción y traducción. Whisper obtiene muy buenos resultados en pruebas de velocidad y precisión respecto a otros modelos, a la vez que es de código abierto.

Es importante mencionar que la inferencia no se realizará utilizando tarjetas gráficas y se usará solo un microprocesador. Esto será un factor extremadamente importante ya que la velocidad de la inferencia es drásticamente mayor al usar una tarjeta gráfica. Por tanto, será importante usar un modelo no demasiado grande para no penalizar demasiado el rendimiento en nuestro caso de uso.

Para nuestro caso será necesario que el modelo nos devuelva la transcripción del audio separando los distintos locutores, señalando los tiempos del audio donde empiezan y donde acaban. Además, también necesitamos que nos devuelva algún factor de confianza de la transcripción, esto será útil a la hora de devolver el texto en la web y que sea fácil identificar las frases que pueden ser más problemáticas marcándolas en un color llamativo.

El modelo Whisper tiene varias maneras de integrarse en el código. La integración más sencilla es a través de la librería transformers de HuggingFace. Esta librería nos simplifica el uso del modelo hasta reducirnos la carga a tan solo el uso de un par de funciones. El problema de este acercamiento es que tenemos que hacer nosotros la identificación de los locutores a mano, utilizando una librería como speaker-diarization.

La manera en la que tendremos que proceder será identificando los locutores y los tiempos que les corresponden usando speaker-diarization. Esta librería da muchos falsos positivos donde devuelve que alguno de los locutores hablan durante apenas unas décimas de segundo. Estos problemas surgen de las pequeñas interrupciones que se producen entre el entrevistador y el entrevistado. Filtraremos todos los fragmentos de tiempo que no sean mayores que un tiempo arbitrario que estimemos oportuno, en este caso 2 segundos. Obtenidos entonces los fragmentos donde hablan cada locutor, pasaremos esos fragmentos del audio al modelo para que haga la inferencia.

Este modelo tiene otra implementación, a través de una librería llamada WhisperX. Esta librería añade algunas herramientas extras que nos serán muy cómodas. La primera herramienta extra que utilizaremos será la detección de actividad de voz (VAD). Que nos ayudará a detectar y reconocer mejor los locutores del audio. Además, también nos ofrece mejoras en precisión a la hora de alinear las palabras con su marca de tiempo en el audio y mejoras en velocidad al utilizar un motor de inferencia más rápido.

5.3.1. Experimentación

Primero vamos a hacer unas pruebas de velocidad para ver la viabilidad de los distintos tamaños que podemos usar en Whisper. Debido a la limitación que tenemos al usar el modelo en un microprocesador, hemos elegido dos variantes pequeñas de Whisper: Small y Medium. También aparecen al principio unas pruebas que se realizaron con una tarjeta gráfica antes de que cambiásemos el objetivo a implementarlo para usar un procesador.

Para estas pruebas vamos a utilizar 2 audios de ejemplo. Ambos audios forman parte de la misma entrevista, uno de ellos es un fragmento de 12 minutos

aproximadamente y el otro es la entrevista completa, con una duración de 2 horas y 12 minutos. Estas pruebas nos servirán para comprobar la velocidad de los modelos a la hora de hacer la transcripción.

Las pruebas se hicieron en dos ordenadores distintos, uno de ellos es un portátil del año 2018 y el otro es un ordenador de sobremesa de 2023. A partir de ahora los llamaremos menos potente y más potente. El ordenador menos potente tiene un procesador Intel Core i7-9750H y el ordenador más potente tiene un procesador AMD 7600x. La gráfica que se utilizó es la gráfica del ordenador menos potente, que es una Nvidia GTX 1650. Esta comparación nos ayudará a ver la importancia de los componentes en la velocidad de ejecución.

Whisper Small		
Configuración	CPU (s)	Gráfica (s)
EJ3 (12M:46S)	960	543
EJ4 (2H:12M:10S)	10109	5738
Whisper large v3 turbo		
Configuración	CPU (s)	Gráfica (s)
EJ3 (12M:46S)	2340	602
EJ4 (2H:12M:10S)	17114	6442
Whisper Medium		
Configuración	CPU (s)	
EJ3 (12M:46S)	1723	-
EJ4 (2H:12M:10S)	14216	-

Tabla 5.1: Tiempos de transcripción con Whisper en el ordenador menos potente

WhisperX Small	
Configuración	CPU (s)
EJ3 (12M:46S), B=4	1074
EJ3 (12M:46S), B=12	733
EJ4 (2H:12M:10S), B=4	7830
EJ4 (2H:12M:10S), B=12	7728
WhisperX Medium	
Configuración	CPU (s)
EJ3 (12M:46S), B=4	2508
EJ3 (12M:46S), B=12	1149
EJ4 (2H:12M:10S), B=4	11605
EJ4 (2H:12M:10S), B=12	12245

Tabla 5.2: Tiempos de transcripción con WhisperX en el ordenador menos potente

Whisper Small	
Tiempo (audio)	CPU (s)
EJ3 (12M:46S)	402
EJ4 (2H:12M:10S)	2928
Whisper Medium	
Tiempo (audio)	CPU (s)
EJ3 (12M:46S)	846
EJ4 (2H:12M:10S)	4238

Tabla 5.3: Tiempos de transcripción con Whisper en el ordenador más potente

WhisperX Small	
Configuración	CPU (s)
EJ3 (12M:46S), B=4	305
EJ3 (12M:46S), B=16	308
EJ4 (2H:12M:10S), B=4	3251
EJ4 (2H:12M:10S), B=16	3103
WhisperX Medium	
Configuración	CPU (s)
EJ3 (12M:46S), B=4	453
EJ3 (12M:46S), B=16	468
EJ4 (2H:12M:10S), B=4	4946
EJ4 (2H:12M:10S), B=16	4474

Tabla 5.4: Tiempos de transcripción con WhisperX en el ordenador más potente

De estas pruebas podemos sacar varias conclusiones. Primero, cabe añadir que de las pruebas que se realizaron con tarjeta gráfica, la mayoría del tiempo se lo dedica a la diarización. En el caso del ejemplo 3, la diarización tarda 448 segundos. En el ejemplo 4, la diarización tarda 4860 segundos. Esto nos deja unos tiempos de inferencia de 1582 segundos usando el tamaño large V3 turbo. Esto implica que si tenemos en cuenta el tiempo de inferencia del modelo medium en el procesador, veremos que es aproximadamente un orden de magnitud más lento para un tamaño menor, lo cual implica menos acierto y más tiempo arreglando errores por parte del usuario. Esto deja claro el enorme potencial que está reservado para la inferencia de los modelos en tarjetas gráficas respecto a su inferencia en procesadores.

También concluimos que el modelo whisper con la diarización de la librería speaker diarization es ligeramente más rápido que el whisperX con la diarización que incluye. Sin embargo, la diferencia de acierto entre la librería de speaker diarization y la diarización de whisperX es demasiado grande, así que no nos compensaría.

Por último, merece la pena también mencionar la opción que nos da WhisperX de cambiar el tamaño de los batches para aumentar la velocidad. Como podemos ver en la tabla, este aumento es ligero, no aumenta la precisión y, además, llega a

aumentar el uso de la memoria RAM hasta los 23 GB aproximadamente cuando lo usamos con whisper medium.

Ahora vamos a hacer una prueba de precisión entre los dos tamaños de Whisper que hemos elegido. Vamos a hacer dos pruebas distintas, la primera de ellas consistirá en comparar el ratio de error de palabras (WER) en un audio muy largo y con condiciones ideales. Hemos elegido la lectura de un libro completo para ello, donde solo hay un locutor y su voz es clara. Vamos a comparar las dos transcripciones usando la librería jiwer de python. Esta prueba nos devuelve que a lo largo de un audio de unas 8 horas y media, el WER del modelo Medium es del 5 % y el WER del modelo Small es del 8 %. Esto parece un cambio pequeño pero en un audio de 8 horas y media, este cambio implica que uno tiene unas 2000 palabras erróneas más que el otro.

El siguiente audio que vamos a utilizar va a ser el fragmento de 12 minutos de la entrevista que utilizamos antes. Puesto que no tenemos un texto con el que comparar, vamos a utilizar el texto del modelo Medium como el correcto y vamos a compararlo con el Small. En este entorno más difícil la diferencia entre los dos tamaños salta hasta el 11 %. Por último, realizamos este mismo experimento pero con la entrevista completa, donde el resultado es el mismo 11 %.

5.3.2. Conclusiones

Podemos concluir que el tamaño de modelo más adecuado para nuestro caso sería el Medium, usando la integración con WhisperX. Alcanzamos esta conclusión en base a las pruebas de rendimiento donde podemos ver que la diferencia de velocidad entre Medium y Small no compensa la diferencia de precisión en entornos más difíciles como una conversación entre dos personas. Además, la herramienta que nos otorga WhisperX para la diferenciación de locutores es más eficaz y acertada, teniendo muchas menos alucinaciones.

5.4. Desarrollo del frontend

En este apartado iremos analizando las distintas ventanas que se desarrollaron para el frontend y los cambios que sufrieron a lo largo del desarrollo.

5.4.1. Pantalla de login

En los diseños iniciales de esta pantalla, el usuario tenía que introducir su correo electrónico y su contraseña. Rápidamente nos dimos cuenta de que el campo del correo electrónico no era necesario. Este campo se cambió por un campo "Nombre", ya que el correo era innecesario, puesto que no hay recuperación de usuario. Esta pantalla es la más simple de todas, ya que simplemente es la puerta de entrada al resto de la web. La imagen de fondo es estática actualmente y cogida de la web de

ADIMA. Se podría cambiar a un carrusel de imágenes en caso de que así lo quisiera la ONG.



Figura 5.1: Pagina de inicio de sesión

5.4.2. Pantalla de bienvenida

La siguiente pantalla es la página de bienvenida. En ella aparece en la barra superior el nombre de ADIMA, el nombre del usuario y un botón a la derecha para cerrar sesión. Esta barra superior será permanente a lo largo de toda la web para poder volver rápidamente a la página inicial haciendo clic en el nombre de ADIMA o para cerrar sesión haciendo clic en su botón correspondiente.

Un poco más abajo tenemos el botón de subir archivo, que al hacerle clic nos abrirá el navegador de archivos para que elijamos un archivo. Este archivo solo puede ser un mp3 y no puede tener el mismo nombre que otro archivo que se haya subido anteriormente. También nos mostrará el peso del archivo y nos permitirá cancelar la elección si nos hemos equivocado. Al subir el archivo, veremos cómo el botón de subir hace una animación de carga y el archivo se añade a la tabla de la carpeta en la que estamos. En caso de que se produzca un error en la subida, el backend falle o ya exista un archivo con ese nombre, aparecerá una alerta que avisa al usuario.

Si seguimos bajando, podemos ver el "breadcrumb" de navegación de las carpetas y un botón para crear una nueva carpeta en la carpeta actual. Este botón abre una pequeña ventana nueva en la que podemos escribir el nombre de nuestra carpeta.

Por último, un poco más abajo tenemos la barra de búsqueda por etiquetas y la tabla en la que se muestran los archivos y las carpetas. En la barra de búsqueda podemos añadir cualquier nombre que le hayamos dado a alguna de las etiquetas que le hayamos añadido a un archivo para filtrar entre los audios. También hay un

botón para elegir si queremos mostrar las carpetas en la tabla o no. Las cabeceras de las columnas de la tabla nos permitirán ordenar los elementos de la tabla por nombre, fecha de subida, estado de transcripción y fecha de edición. Todo esto facilita que el usuario pueda encontrar el audio que busca sin problemas. De momento todos los usuarios pueden ver todos los archivos, pero se podría cambiar y añadir un sistema jerárquico en el que algunos usuarios tengan más privilegios de visualización o edición que otros. Por terminar, queda hablar de la columna de acciones, donde podemos ver el botón de eliminar que usaremos para borrar el audio o la carpeta que queramos y a su izquierda encontramos el botón de etiquetas. Este botón nos abrirá una ventana en la que podremos editar las etiquetas de un archivo, añadiendo o quitando las que queramos hasta un máximo de 5.

Esta pantalla cambió mucho estéticamente respecto al primer "mockup" que se hizo de ella. Esto es ya que en el "mockup" no había ningún tipo de ordenación por carpetas, ni había una tabla que mostrase los datos importantes de manera rápida y visual.

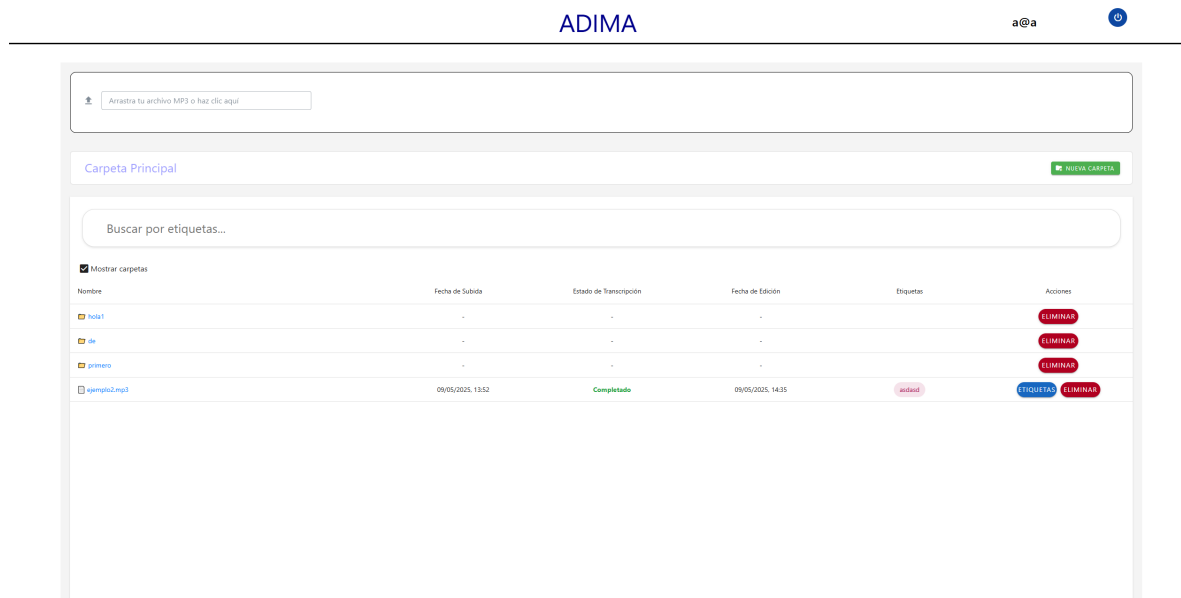


Figura 5.2: Pantalla de bienvenida

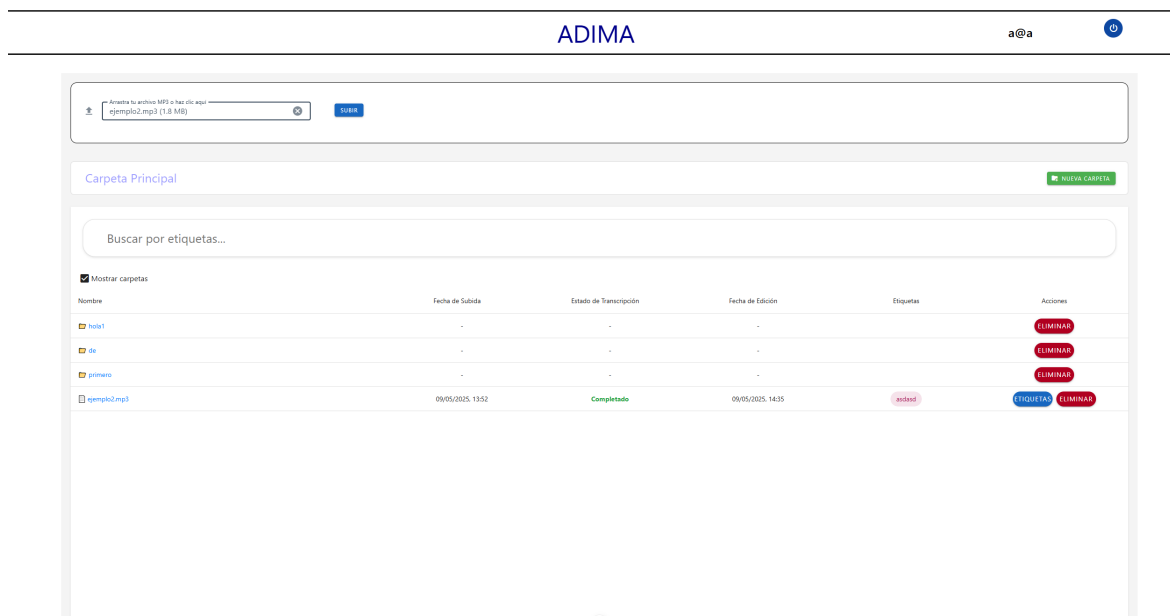


Figura 5.3: Pantalla de bienvenida con archivo preparado para subir

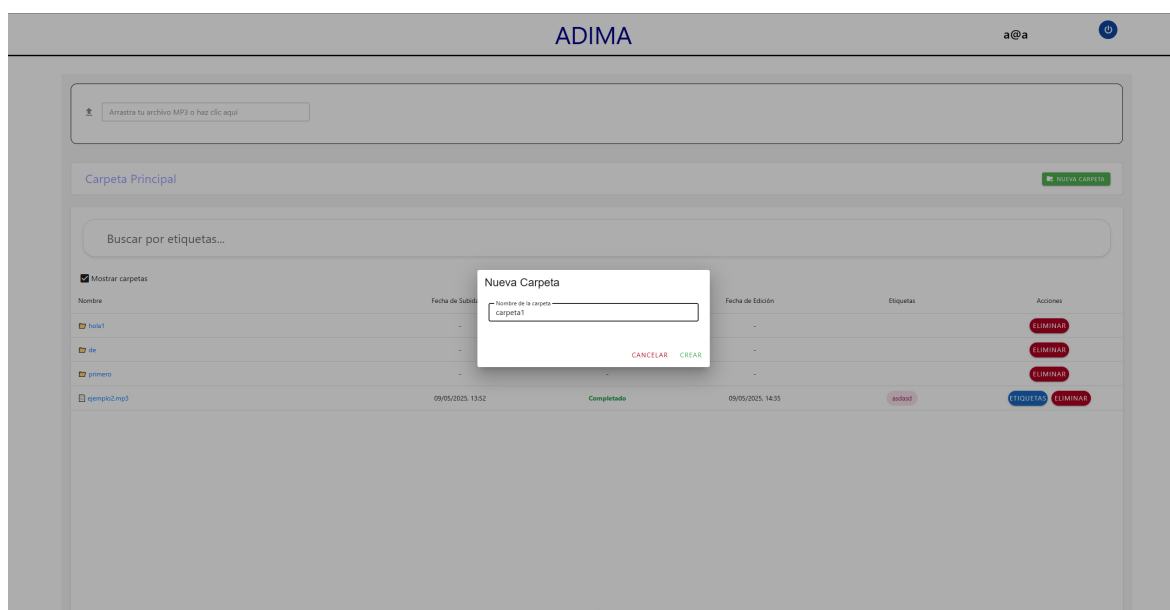


Figura 5.4: Pantalla de bienvenida con menú para crear carpeta

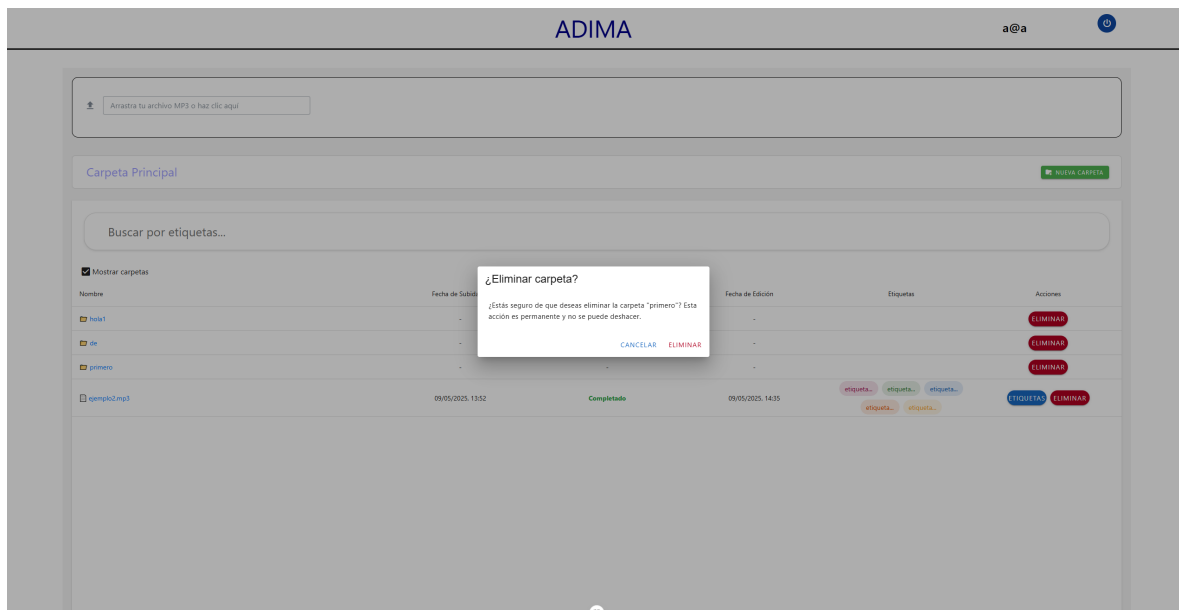


Figura 5.5: Pantalla de bienvenida con menú para eliminar carpeta

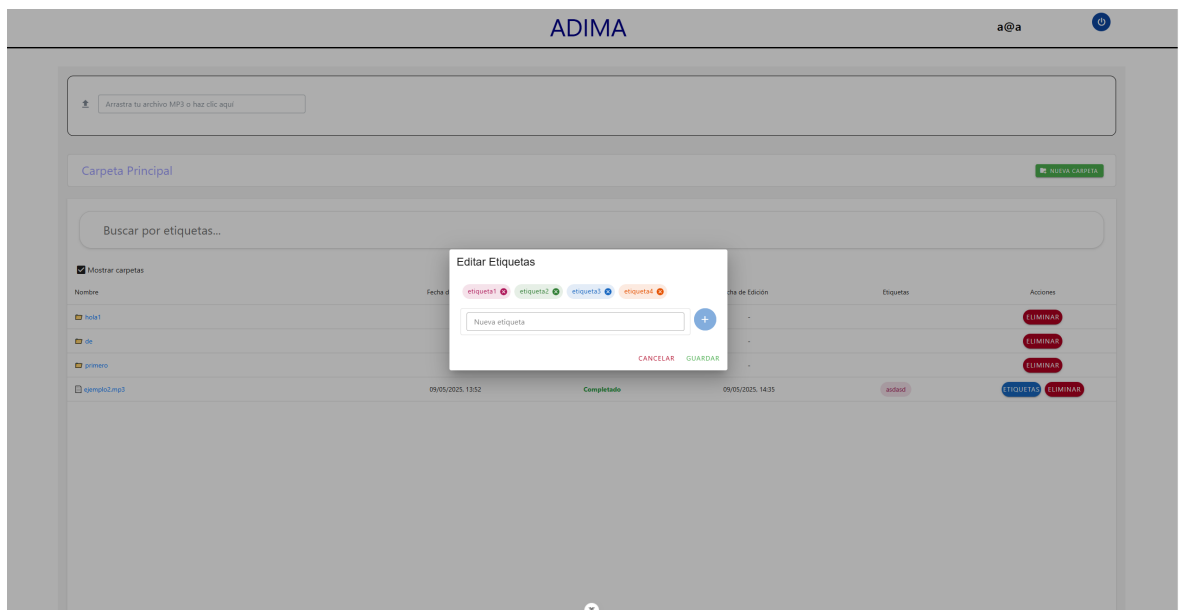


Figura 5.6: Pantalla de bienvenida con menú para añadir etiquetas

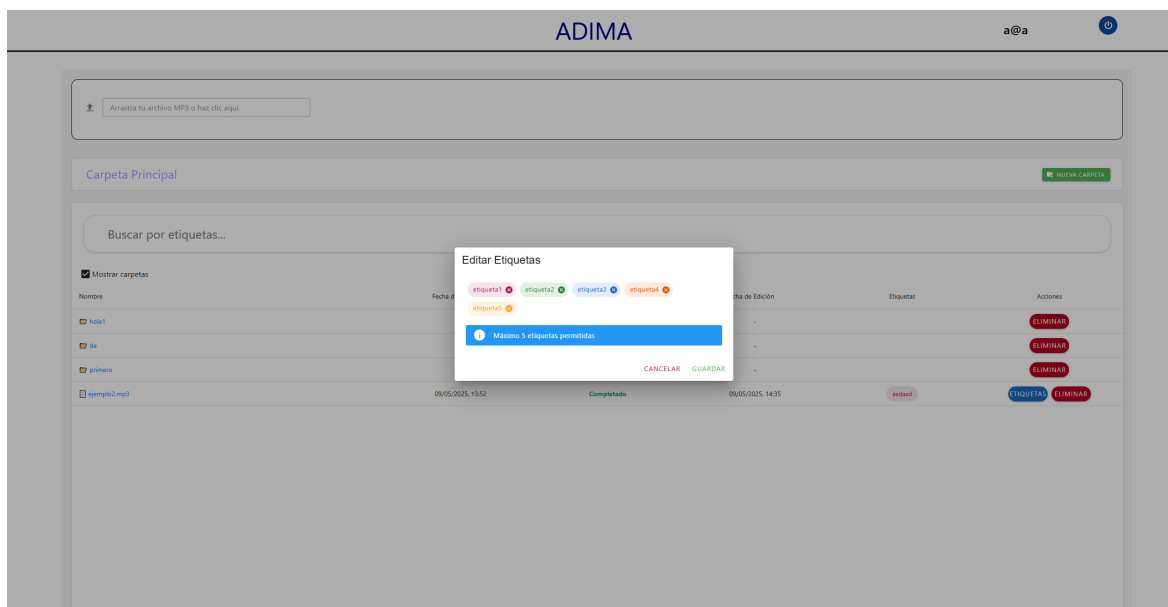


Figura 5.7: Pantalla de bienvenida con menú para añadir etiquetas completo

5.4.3. Pantalla de transcripción

Al hacer clic en el nombre de un archivo cuya transcripción ya esté completada podemos acceder a la pantalla de transcripción. En esta pantalla encontraremos en la parte superior una flecha para volver a la página de bienvenida, una barra para buscar en el texto de la transcripción y dos botones. Estos botones son los botones de exportar y activar edición. El primero despliega un menú con las opciones de exportar a docx, pdf y txt. Esta herramienta es imprescindible para que los trabajadores de la ONG puedan extraer el texto como ellos quieren. El botón de activar edición nos avisará de que estamos a punto de editar el texto, y que esta acción es irreversible. Si confirmamos la elección, este botón no aparecerá más para este archivo.

Un poco más abajo tenemos la transcripción que nos ha devuelto el ASR dividida en frases. En la parte superior izquierda de la caja que contiene cada frase encontramos el número que se le asigna a cada locutor. Este número es editable y cada locutor tiene el fondo de un color distinto. Esto ayuda a que podamos visualizar rápidamente quién está hablando y comprobarlo de manera sencilla. Un poco más abajo tenemos la marca de tiempo de cada frase. Hacer clic en esta marca de tiempo hace que el audio se traslade hasta allí automáticamente. Para visualizar de manera más cómoda qué frase está sonando ahora mismo, el fondo del cuadro cambia de color y aparece una barra verde en el lateral izquierdo del cuadro, además de que el foco del scroll cambia para dejar ese cuadro en el centro. Si vamos hasta el texto podemos ver que tiene una pequeña barra de color que varía entre roja, naranja, amarilla y verde. Esta barra indica la certeza que devuelve el modelo de esa frase. También aparece esta certeza como porcentaje en la parte inferior derecha del cuadro.

Si hemos activado la edición del archivo podremos alterar el texto de los

cuadros. Al terminar de editar el texto es necesario que el usuario presione la tecla enter. Cuando el usuario hace esto el texto se pone en negrita, desaparece la barra de color que indica la certeza y además el porcentaje de la esquina inferior derecha se sustituye por una marca de verificación.

Por último, en la parte de abajo de esta pantalla tenemos la parte del reproductor de audio. En ella podemos ver el nombre del archivo, podemos avanzar o retroceder en el audio con los botones, parar o reanudar la reproducción, subir o bajar el volumen y guardar el archivo cuando se haya editado algún cuadro. Cuando todos los cuadros hayan sido revisados, el estado de la transcripción del archivo cambiará a Revisado. Esto facilita la rápida identificación de los archivos que ya han sido revisados y que pueden ser exportados.

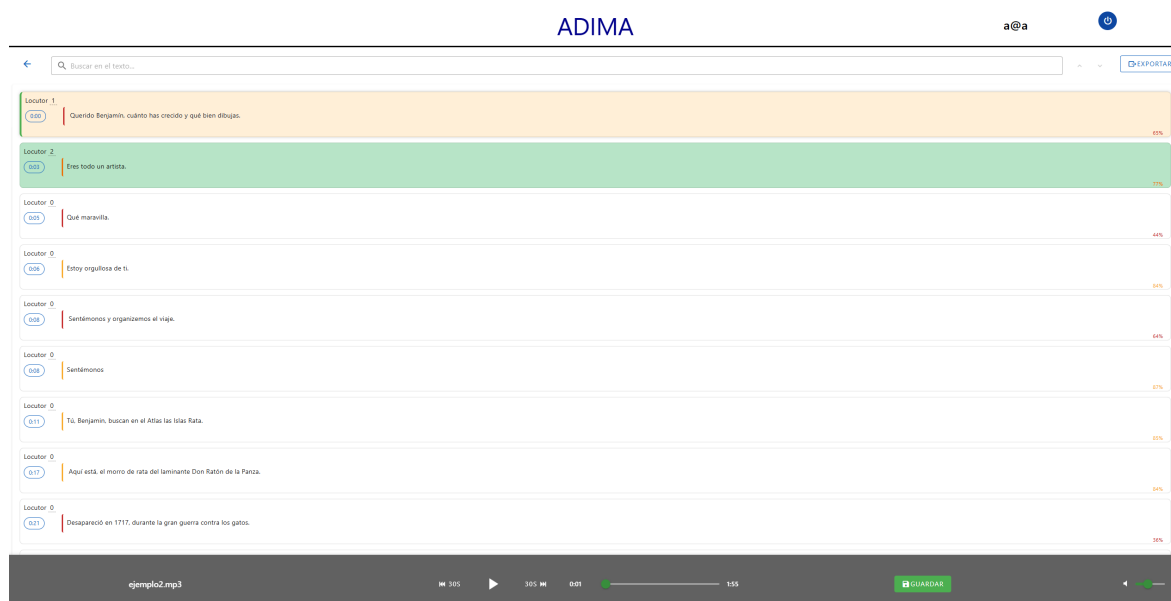


Figura 5.8: Pantalla de transcripción

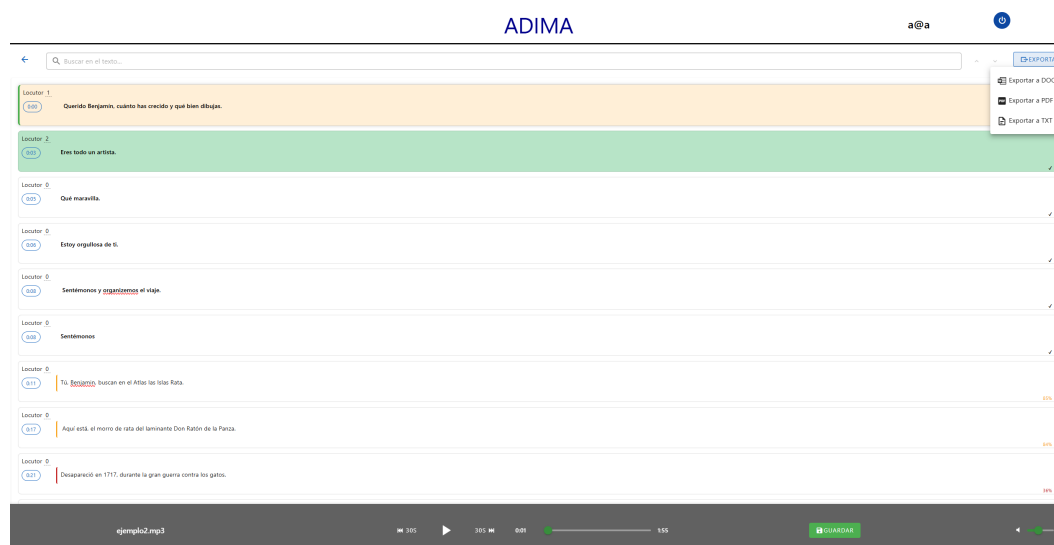


Figura 5.10: Pantalla de transcripción con menú de exportación

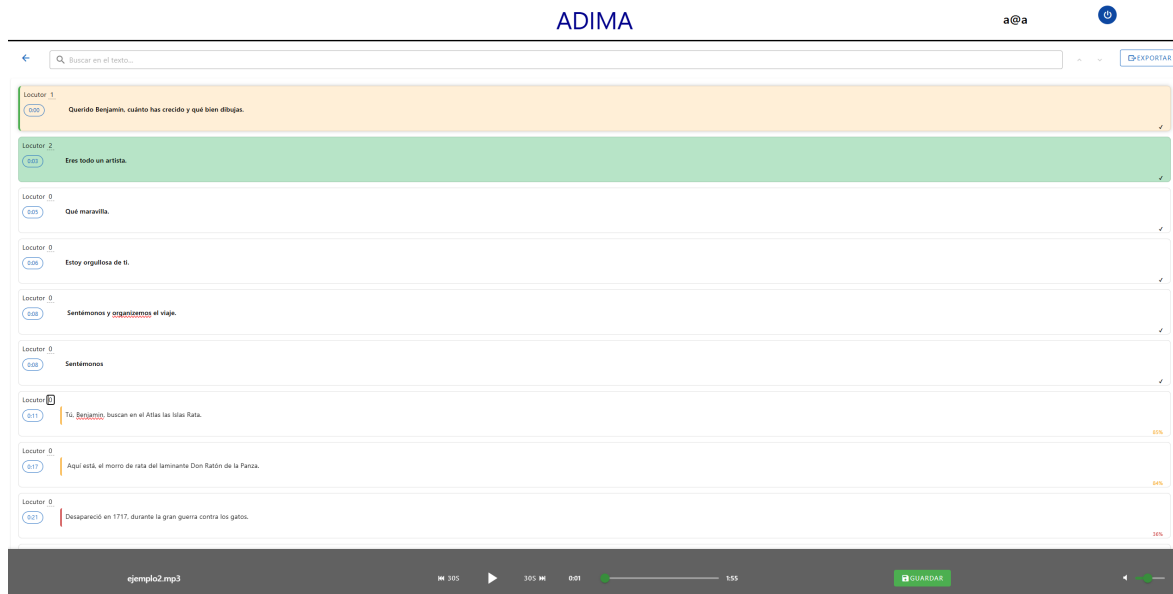


Figura 5.9: Pantalla de transcripción con checkmarks

5.5. Desarrollo del backend

En este capítulo hablaremos sobre el desarrollo del backend de la aplicación. Y cómo se abordaron los distintos problemas que surgieron durante su desarrollo.

5.5.1. Modelos

La primera parte que trataremos del backend son los modelos. En Django, los modelos representan la estructura de los datos de la aplicación y permiten la interacción con la base de datos de forma abstracta y sencilla gracias a su sistema ORM (Object-Relational Mapping).

Mediante el uso de clases Python, los modelos definen las tablas de la base de datos, así como los campos y relaciones entre ellas. Cada clase de modelo se traduce en una tabla, y cada atributo de la clase corresponde a una columna en dicha tabla. Esto permite trabajar con datos utilizando código Python sin necesidad de escribir consultas SQL manualmente, lo que mejora la legibilidad, el mantenimiento y la portabilidad del código.

Los modelos se crearon siguiendo el diseño del diagrama de clases que se hizo anteriormente, aunque acabó sufriendo cambios. A continuación veremos una lista de los modelos que se crearon en la aplicación:

- **Usuario:** Por simplicidad utilizamos el usuario base que ofrece Django: "BaseUserManager". Aunque realmente no necesitamos tantos campos, solo nombre y contraseña para la estructura actual.
- **Carpeta:** Todas tienen un nombre y un padre, a excepción de la carpeta principal, que es la única que no tiene padre.

- **Archivo:** Representan los archivos de audio que sube el usuario. Guarda la fecha de subida y la duración total del audio.
- **Transcripción:** Esta clase indica el estado de la transcripción. Además, almacena también la confianza media, las etiquetas, la fecha de modificación y si ha sido editada.
- **Tiempo_Texto:** Estas clases están relacionadas con una sola transcripción. Es la clase que utilizamos para indicar el texto, el locutor, las marcas de tiempo de cada frase, la confianza y si ha sido revisada.

Por tanto, el diagrama de clases que nos queda es distinto al que teníamos al principio.

Ahora recorreremos los cambios que se han producido entre el primer diseño y el diseño final.

1. **Eliminación del correo del usuario.**
2. **Eliminación del formato del archivo.**
3. **Movida fecha actualización de archivo a transcripción.**
4. **Añadido el campo `task_id` a archivo.**
5. **Añadido el campo `archivo` a archivo, mostrando la ruta donde se guarda.**
6. **Añadido fecha transcripción a transcripción.**
7. **Añadido el campo `etiquetas` a transcripción.**
8. **Añadido el campo `modificado` a transcripción.**
9. **Añadido el campo `locutor` a `tiempo_texto`.**
10. **Añadido el campo `confianza` a `tiempo_texto`.**
11. **Añadido el campo `revisado` a `tiempo_texto`.**

5.5.2. Serializadores

En Django REST Framework, los serializadores son un componente fundamental para convertir los datos complejos (como objetos de base de datos) a formatos nativos de Python que puedan ser fácilmente renderizados como JSON, y viceversa. Actúan como el puente entre la lógica del backend y los datos que se intercambian con el frontend.

En nuestra aplicación, los serializadores permiten estructurar, validar y transformar la información que viaja entre los endpoints de la API y los componentes de la interfaz de usuario. Cada entidad principal del sistema (usuarios, archivos de audio, transcripciones) cuenta con su correspondiente serializador, encargado no solo de definir los campos que deben exponerse o aceptarse, sino también de realizar comprobaciones automáticas para evitar inconsistencias o datos inválidos.

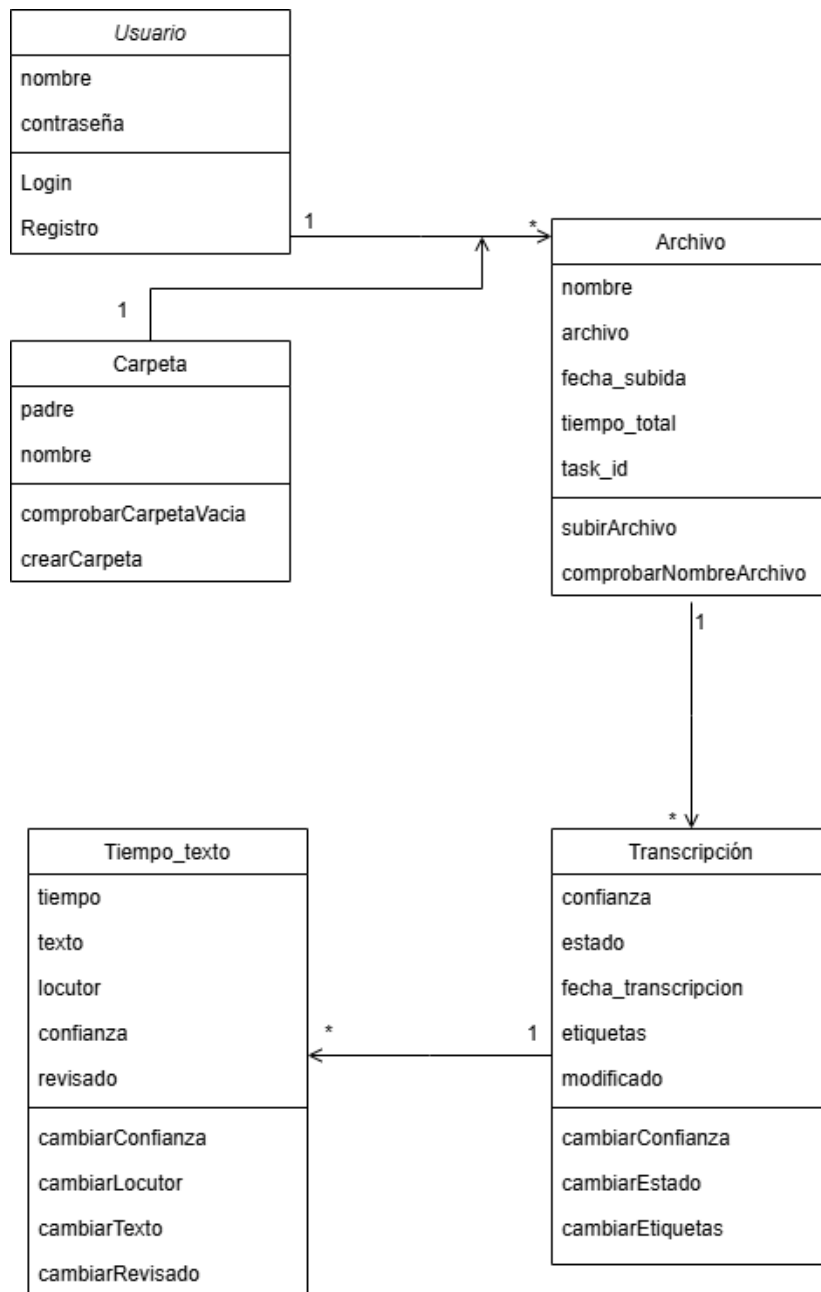


Figura 5.11: Diagrama de clases

A continuación, describimos los principales serializadores utilizados en el proyecto y algunas decisiones específicas de implementación.

- **UsuarioSerializer:** Permite la serialización de los datos del modelo `Usuario`, incluyendo únicamente los campos necesarios para la autenticación básica (ID, nombre de usuario y contraseña).
- **ArchivoSerializer:** Gestiona la serialización de archivos de audio subidos por el usuario. Además de incluir información como nombre, fecha de subida y duración, añade un campo calculado llamado `estado_transcripcion`, que extrae el estado actual de la transcripción asociada (si existe). Este enfoque permite mostrar al usuario un resumen actualizado del progreso del procesamiento.
- **TranscripcionSerializer:** Serializa las transcripciones generadas por el modelo ASR. Formatea la fecha de transcripción y permite el envío completo de los campos definidos en el modelo `Transcripcion`, facilitando la visualización, edición y exportación de los resultados.
- **Tiempo_TextoSerializer:** Encargado de representar el texto, resultado del análisis de audio. Cada entrada incluye marcas de tiempo y texto asociado, lo que permite su representación sincrónica en el reproductor de audio de la aplicación.
- **CarpetaSerializer:** Permite estructurar los archivos dentro de carpetas virtuales, de modo que los usuarios puedan organizar sus archivos subidos por temas o entrevistas. Serializa todos los campos definidos en el modelo `Carpeta`.
- **ArchivoTranscripcionSerializer:** Extiende el serializador de archivos incluyendo el objeto de transcripción completo como un campo anidado. Resulta útil para mostrar toda la información relevante de un archivo en una única petición API.
- **TranscripcionTiempoTextoSerializer:** Serializa una transcripción con su correspondiente conjunto de entradas temporales de texto. Utiliza el campo `source` para acceder a la relación inversa entre la transcripción y sus fragmentos. Incluye un campo booleano `modificado`, que permite al frontend saber si el usuario ha editado el contenido de forma manual.

Gracias a estos serializadores, el backend de la aplicación es capaz de traducir eficientemente los datos entre la base de datos y el cliente. Esto permite una comunicación fluida entre el frontend y el backend, garantizando que toda la información relevante sobre usuarios, archivos y transcripciones se transmita de forma clara, segura y estructurada.

5.5.3. Endpoints y estructura de la API REST

La comunicación entre el frontend (SPA desarrollada en Vue) y el backend (Django) se realiza a través de una API REST, implementada con Django REST

Framework. Esta API está estructurada en múltiples rutas (endpoints), cada una asociada a un recurso principal del sistema: usuarios, archivos, transcripciones, tiempos de texto y carpetas. Todos los endpoints se comunican en formato JSON, utilizan los métodos HTTP estándar (GET, POST, PUT, DELETE) y están protegidos mediante autenticación basada en JWT.

■ **Usuarios:**

- POST /register/ — Registra un nuevo usuario. Aplica validación del nombre y encripta la contraseña.
- POST /login/ — Inicia sesión, descifra la contraseña recibida mediante RSA y devuelve los tokens JWT.
- GET /usuario/list/ — Lista todos los usuarios registrados.
- GET/PUT/DELETE /usuario/<id>/ — Consulta, actualiza o elimina un usuario específico.
- GET /public-key/ — Devuelve la clave pública usada por el frontend para cifrar contraseñas.
- POST /refresh/ — Refresca el token de acceso JWT mediante el token almacenado en cookies.

■ **Archivos:**

- POST /archivo/ — Permite a un usuario autenticado subir un archivo de audio. Inicia una tarea de transcripción usando Celery y WhisperX.
- GET /archivos/ — Devuelve todos los archivos. Puede filtrar por carpeta.
- DELETE /archivo-delete/ — Elimina un archivo, su transcripción asociada y detiene la tarea si está en curso.

■ **Transcripciones:**

- GET /transcripcion/<id>/ — Devuelve la transcripción asociada al archivo, con los tiempos por palabra.
- POST /transcripcion/ — Marca una transcripción como completada y actualiza su fecha.
- PUT /transcripcion/ — Permite modificar etiquetas asociadas a una transcripción.

■ **Tiempo de texto:**

- POST /tiempo-texto/ — Recibe una lista de fragmentos de transcripción con marcas de tiempo y locutores. Si todos los fragmentos son revisados, marca la transcripción como “Revisada”.

■ **Carpetas:**

- GET /carpeta/ — Lista las carpetas hijas de la raíz o de una carpeta específica.

- POST /carpeta/ — Crea una nueva carpeta, asignándole opcionalmente una carpeta padre.
- PUT /carpeta/<id>/ — Permite renombrar o mover una carpeta.
- DELETE /carpeta/<id>/ — Elimina una carpeta, siempre que no tenga archivos ni subcarpetas.

■ Exportación:

- POST /exportar/docx/ — Exporta la transcripción en formato .docx.
- POST /exportar/pdf/ — Exporta la transcripción en formato PDF.

La API implementa una arquitectura clara y coherente, lo que permite al frontend interactuar con los distintos recursos del sistema de forma estructurada. El uso de serializadores garantiza que cada petición sea validada y transformada correctamente antes de llegar a la base de datos. Además, la protección mediante JWT y los permisos 'IsAuthenticated' aseguran que los endpoints críticos solo estén disponibles para usuarios autenticados.

5.5.4. Autenticación y permisos

La aplicación implementa un sistema de autenticación basado en **JSON Web Tokens (JWT)**, un método moderno y ampliamente utilizado para mantener sesiones seguras sin necesidad de almacenar información en el servidor. Cuando un usuario se identifica correctamente mediante el formulario de inicio de sesión, el backend genera un token JWT que es devuelto al cliente. Este token se adjunta en las cabeceras de las peticiones posteriores y permite verificar la identidad del usuario de forma eficiente y segura.

Una característica destacable del sistema es el uso de **cifrado asimétrico RSA** para proteger las contraseñas en tránsito. El frontend obtiene la clave pública del servidor mediante el endpoint /public-key/, y utiliza esta clave para cifrar la contraseña antes de enviarla al backend. El backend, que conserva la clave privada correspondiente, descifra la contraseña y procede a verificarla. Este mecanismo añade una capa adicional de seguridad frente a posibles interceptaciones, especialmente en el contexto de entornos de prueba sin HTTPS.

Además del uso de JWT, el sistema incorpora medidas de seguridad adicionales:

- **Protección CSRF:** Aunque las SPA con JWT no se ven afectadas por ataques CSRF en el mismo grado que las sesiones tradicionales, se configuran cabeceras y tokens para proteger endpoints sensibles cuando se accede desde navegadores.
- **CORS (Cross-Origin Resource Sharing):** Se habilita selectivamente el acceso entre dominios (por ejemplo, entre frontend local y backend en desarrollo) mediante configuración específica en Django. Esto permite la comunicación entre los módulos del sistema sin comprometer la seguridad.

- **Permisos por endpoint:** Se usa la clase `IsAuthenticated` como restricción general para todos los endpoints de la API que requieren sesión activa. Esto evita que usuarios no autenticados puedan consultar, modificar o eliminar información protegida.

Este sistema de autenticación asegura un acceso controlado, cifrado y verificable para los distintos usuarios de la aplicación, garantizando la integridad y privacidad de los datos procesados.

5.5.5. Exportación de transcripciones

Una vez que las transcripciones han sido revisadas y validadas por los usuarios, la aplicación ofrece la posibilidad de **exportarlas en distintos formatos** para su uso fuera del sistema. Esta funcionalidad es especialmente útil en el contexto de la ONG, ya que permite archivar entrevistas, compartirlas con terceros o incorporarlas a informes.

El backend implementa dos rutas principales para la exportación:

- `POST /exportar/docx/`: Genera un documento en formato Microsoft Word (.docx) a partir de los datos enviados desde el frontend. Este formato permite conservar el formato estructurado de la transcripción, con los tiempos y locutores alineados a lo largo del documento. Se utiliza la biblioteca `python-docx` para componer dinámicamente el archivo.
- `POST /exportar/pdf/`: Alternativamente, se permite la exportación en formato PDF, útil para archivado o distribución solo de lectura.

En ambos casos, el contenido del archivo es generado en el servidor y devuelto como respuesta descargable. La exportación incluye:

- Información del archivo original (nombre, duración, fecha).
- Identificación de locutores por línea.
- Tiempos de inicio por segmento.
- Texto.

Este mecanismo de exportación complementa la funcionalidad de edición y revisión, y cierra el ciclo completo de trabajo dentro de la aplicación, desde la carga del audio hasta la obtención del documento final transcrito.

5.6. Gestión de colas

En una aplicación web orientada a la productividad, como la desarrollada en este proyecto, la capacidad de gestionar tareas de forma asíncrona y eficiente resulta fundamental. En este caso particular, las tareas que requieren mayor carga computacional (como la inferencia del modelo de transcripción WhisperX), deben ejecutarse en segundo plano, sin bloquear la interacción del usuario con el resto de

la aplicación. Para ello, se ha implementado un sistema de gestión de tareas en segundo plano a través de la integración de **Celery** y **Redis**.

Celery es una librería escrita en Python para la gestión de colas de tareas asíncronas, que permite distribuir cargas de trabajo entre procesos separados llamados **workers**. Estos workers se encargan de ejecutar las tareas más costosas (en este caso, la inferencia del modelo de ASR) sin afectar al rendimiento del servidor principal de Django. Por su parte, **Redis** es un sistema de almacenamiento en memoria clave-valor que, entre otros usos, puede actuar como *message broker*, es decir, como intermediario para el envío y recepción de tareas entre Django y Celery.

En nuestra aplicación, cada vez que un usuario sube un archivo de audio, se genera automáticamente una tarea que se añade a la cola. Celery, a través de su worker, recibe la tarea y la ejecuta en segundo plano: esto incluye la carga del modelo WhisperX, la transcripción del audio y la posterior actualización del estado del archivo y la base de datos. Redis actúa como el canal de comunicación entre el backend de Django y los workers de Celery, gestionando en tiempo real qué tareas están pendientes, en ejecución o finalizadas.

A pesar de la robustez conceptual de esta arquitectura, la implementación práctica se ha enfrentado a una serie de limitaciones técnicas importantes, derivadas principalmente del entorno de desarrollo. En concreto, el sistema operativo Windows (utilizado durante el desarrollo) presenta múltiples incompatibilidades con Celery, especialmente en lo relativo al uso de múltiples workers simultáneos. Aunque Celery puede ejecutarse en Windows en modo experimental, su funcionamiento no es estable y carece de soporte completo. Esto ha impedido realizar pruebas más avanzadas de concurrencia y escalado, lo cual sería necesario para evaluar el comportamiento del sistema en situaciones con múltiples usuarios activos y varias tareas de transcripción en paralelo.

Para suplir parcialmente estas limitaciones, se ha utilizado una imagen Docker oficial de Redis, ejecutada como contenedor durante el desarrollo, lo cual ha permitido mantener una arquitectura funcional de pruebas. Sin embargo, no se ha podido dockerizar todo el entorno (backend, workers, frontend) ni se ha llegado a desplegar un sistema de gestión completo en un servidor de producción, debido a restricciones de tiempo y recursos. Este hecho se reconoce como una de las limitaciones más importantes del proyecto en su estado actual, aunque se han sentado las bases necesarias para que, en un futuro, se pueda completar una implementación más escalable, robusta y orientada a producción.

En resumen, aunque la gestión de colas mediante Celery y Redis ha sido integrada de forma funcional en la arquitectura de la aplicación, no se ha podido explotar su potencial completo debido a las restricciones del entorno de desarrollo. Aun así, la estructura lógica está preparada para que pueda evolucionar hacia un sistema distribuido real, capaz de gestionar múltiples tareas concurrentes en un entorno controlado y escalable.

5.7. Futuras extensiones

En esta sección se describen posibles mejoras del sistema que podrían incorporarse en futuras versiones del proyecto, con el objetivo de aumentar la precisión de la transcripción y mejorar el rendimiento.

5.7.1. Fine-tuning de Whisper

Una de las vías principales de mejora consiste en realizar un *fine-tuning* del modelo Whisper sobre un listado específico de audios de entrevistas de la ONG. Aunque Whisper ofrece una excelente capacidad *zero-shot*, afinar el modelo con ejemplos representativos del dominio (acento, terminología, ruido de fondo típico) permitiría reducir aún más la tasa de error de palabras (WER) y mejorar la consistencia en la identificación de locutores. El proceso de fine-tuning requeriría:

- Recolectar y anotar manualmente un subconjunto de audios de entrevistas.
- Adaptar el código de entrenamiento de Whisper (usando la librería transformers de Hugging Face) para recalibrar pesos en capas superiores.
- Validar el nuevo modelo mediante un conjunto de pruebas controladas y medir la mejora en WER y en precisión de diarización.

Este enfoque permitiría que la aplicación aprendiera términos propios de la ONG (nombres, acrónimos, expresiones recurrentes) y optimizara la calidad de la transcripción en condiciones reales de uso.

5.7.2. Infraestructura de cómputo dedicada

Para ejecutar el modelo Whisper large (1.55 B parámetros) con buena velocidad, es recomendable disponer de una GPU con al menos **12 GB de VRAM**. De esta forma, se podría cargar el modelo completo en memoria de vídeo y aprovechar la inferencia acelerada por CUDA:

- Modelos medium o small pueden ejecutarse en GPUs de 8 GB, pero la large ofrece la mayor precisión.
- Una tarjeta gráfica de gama media-alta (por ejemplo, NVIDIA RTX 3060 12 GB) permite un tiempo de inferencia cercano a $1\times$ tiempo real o mejor, reduciendo la latencia de transcripción.
- Para cargas concurrentes, se podrían emplear dos GPUs en el mismo servidor o balancear las tareas entre varias máquinas con Celery.

5.7.3. Estimación de presupuesto

Como referencia, un equipo de sobremesa con las siguientes características permitiría cubrir las necesidades de cómputo:

- **Carcasa y fuente de alimentación** (750 W, 80 Plus Bronze): 125 €
- **Placa base** AM5 o compatible con el procesador: 160 €
- **Procesador** Ryzen 5 7600X o equivalente: 190 €
- **Memoria RAM** 32 GB DDR5: 100 €
- **Almacenamiento** SSD NVMe 2 TB: 130 €
- **Tarjeta gráfica** NVIDIA RTX 3060 12 GB: 300 €
- **Refrigeración** CPU y ventiladores adicionales: 40 €
- **Otros componentes y montaje**: 50 €
- **Licencia de windows**: 5 €

Total estimado: 1100 €

Este presupuesto incluye el hardware necesario para ejecutar el modelo Whisper large en local con un rendimiento óptimo. Esto permitiría que la transcripción fuera mucho más precisa y aún más rápida, reduciendo así la carga de trabajo para los trabajadores.

5.7.4. Dockerización y despliegue

Para garantizar un despliegue consistente, reproducible y sencillo de toda la aplicación (frontend, backend, worker de Celery, broker Redis y base de datos), deberíamos empaquetar cada componente dentro de su propio contenedor Docker y orquestarlos mediante Docker Compose. A grandes rasgos, los pasos a seguir serían los siguientes:

1. **Definir contenedores independientes:** Cada parte de la aplicación (el servidor Django, el worker de Celery, Redis, PostgreSQL y la SPA en Vue) se ejecuta en su propio contenedor, con las dependencias y configuraciones aisladas.
2. **Configurar la red interna:** Se crea una red virtual privada para que los contenedores se comuniquen entre sí (por ejemplo, Django accediendo a la base de datos o a Redis) sin exponer puertos innecesarios al exterior.
3. **Gestionar variables de entorno y secretos:** Todos los parámetros sensibles (credenciales de base de datos, claves secretas de Django, tokens de WhisperX) se cargan en tiempo de ejecución desde un fichero de entorno o un gestor de secretos, sin quedar embebidos en el código.
4. **Orquestar con Docker Compose:** Un único archivo de configuración describe los servicios, sus dependencias, volúmenes de datos persistentes y los puertos a exponer. Con un solo comando se inicia, actualiza o detiene todo el sistema.

5. **Automatizar despliegues y actualizaciones:** Al encapsular toda la lógica de construcción en Dockerfiles, basta reconstruir y relanzar los contenedores para aplicar cambios de código o de configuración sin riesgo de roturas en el servidor de producción.

6. Conclusiones

El desarrollo de la inteligencia artificial está calando cada vez más en nuestra vida cotidiana, abriendo nuevas posibilidades para mejorar la eficiencia en tareas rutinarias y potenciar actividades de todo tipo. Este ritmo de avance vertiginoso en el campo de la IA trae consigo enormes beneficios, pero también plantea retos éticos y de regulación que deben abordarse con responsabilidad. En este contexto, proyectos como el presente demuestran cómo la tecnología puede ponerse al servicio de causas nobles, al tiempo que subrayan la necesidad de un desarrollo ágil pero cuidadoso.

A lo largo de este Trabajo de Fin de Grado hemos comprobado los retos y aciertos de integrar un sistema de transcripción automática local basado en WhisperX en un entorno semi-real de trabajo para ADIMA. Uno de los principales desafíos técnicos ha sido la diferencia de rendimiento entre la inferencia en CPU y en GPU. Mientras que la ejecución en procesadores sin aceleración gráfica resultó excesivamente lenta (mientras más pequeño era el modelo, peor era la precisión y más tiempo era necesario para corregir errores manualmente), el uso de una GPU potente podría reducir los tiempos de inferencia y sacarle todo el partido a versiones más grandes de WhisperX. Esta experiencia ha puesto de manifiesto la importancia de disponer de recursos de cómputo adecuados cuando se trabaja con modelos de inteligencia artificial aunque no sean de gran tamaño o alta complejidad.

El éxito de WhisperX en este proyecto ha sido notable. Gracias a su capacidad de generar marcas de tiempo con gran precisión y su funcionalidad de diarización de hablantes, hemos proporcionado a los usuarios una herramienta práctica y precisa para revisar y corregir transcripciones de entrevistas.

La primera presentación de la aplicación ante los trabajadores de la ONG fue todo un hito. El hecho de contar con usuarios reales que valoraron positivamente la interfaz, la velocidad de respuesta y la facilidad de edición de textos confirmó la viabilidad y utilidad de la solución. El visto bueno de quienes van a utilizarla en su día a día refuerza la relevancia de este proyecto y avala la elección de las tecnologías empleadas.

Sin embargo, la falta de tiempo nos impidió completar algunas mejoras importantes. Lamentamos no haber podido dockerizar toda la aplicación, un paso que habría simplificado enormemente el despliegue en servidores de la ONG y acelerado la puesta en producción. Asimismo, la falta de reuniones periódicas con los responsables de la organización impidió definir con claridad el flujo de creación de nuevos usuarios: todavía no hemos implementado un mecanismo amigable para que los administradores añadan cuentas desde la interfaz, ni limitaciones finas que confinen el acceso a archivos y transcripciones al usuario que los sube. En la actualidad, cualquier usuario puede ver todas las transcripciones, algo que debería revisarse para garantizar la privacidad y el control de la información sensible.

En conjunto, el proyecto puede considerarse un éxito: se ha construido una aplicación funcional, con un backend robusto en Django, un frontend intuitivo en Vue, y un pipeline de transcripción fiable con Celery, Redis y WhisperX. Quedan por delante mejoras de usabilidad y despliegue, pero la base sentada y la aprobación de la ONG demuestran que este trabajo aporta valor real y abre la puerta a futuras extensiones y desarrollos aún más ambiciosos.

Bibliografía

- [1] ADIMA. Adima · promoviendo el buen trato a la infancia y adolescencia – es una organización sin ánimo de lucro promocionar el buen trato y la atención a la infancia y la adolescencia. URL <https://adima.com.es/>.
- [2] Martín Ortiz-Domínguez. Redes neuronales artificiales. *Ingenio y Conciencia Boletín Científico de la Escuela Superior Ciudad Sahagún*, 12:38–44, 01 2025. doi: 10.29057/escs.v12i23.14132.
- [3] Marco Cruz-Rocca, Williams Muñoz-Robles, and Melquiades Trinidad-Malpartida. Análisis de la aplicación de redes neuronales convolucionales en el área de la visión artificial. *Investigación Valdizana*, 18:e2105, 03 2024. doi: 10.33554/riv.18.1.2105.
- [4] Ian Smalley Mesh Flinders. ¿qué es la inferencia de la ia? — ibm. URL <https://www.ibm.com/es-es/think/topics/ai-inference>.
- [5] Aafaq Khan Arsalan Shahid Venkatesh Balavadhani Parthasarathy, Ahtsham Zafar. [2408.13296] the ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities, 10 2024. URL <https://arxiv.org/abs/2408.13296>.
- [6] Dipesh Gyawali. Untitled, 12 2023. URL <https://arxiv.org/pdf/2309.02521>.
- [7] OpenAI. Introducing whisper — openai, 09 2022. URL <https://openai.com/index/whisper/>.
- [8] m bain. Github - m-bain/whisperx: Whisperx: Automatic speech recognition with word-level timestamps (& diarization), 05 2023. URL <https://github.com/m-bain/whisperX>.
- [9] META. React. URL <https://es.react.dev/>.
- [10] Vue.js. Vue.js - the progressive javascript framework. URL <https://vuejs.org/>.
- [11] Spring. Spring — home. URL <https://spring.io/>.
- [12] FastApi. Fastapi. URL <https://fastapi.tiangolo.com/>.
- [13] Django. The web framework for perfectionists with deadlines — django. URL <https://www.djangoproject.com/>.
- [14] Vosk. Vosk offline speech recognition api. URL <https://alphacephei.com/vosk/>.

- [15] mozilla. Mozilla/deepspeech: Deepspeech is an open source embedded (offline, on-device) speech-to-text engine which can run in real time. URL <https://github.com/mozilla/DeepSpeech>.
- [16] Ticiana Capris, Pedro Melo, Nuno Garcia, Ivan Pires, and Eftim Zdravevski. Comparison of sql and nosql databases with different workloads: Mongodb vs mysql evaluation. *2022 International Conference on Data Analytics for Business and Industry (ICDABI)*, 02 2023. doi: 10.1109/ICDABI56818.2022.10041513.
- [17] Celery. Celery - distributed task queue. URL <https://docs.celeryq.dev/en/stable/>.
- [18] Redis. Redis - la plataforma de datos en tiempo real. URL <https://redis.io/es/>.
- [19] Microsoft. Visual studio code - code editing. redefined, 11 2021. URL <https://code.visualstudio.com/>.
- [20] Github. Github · build and ship software on a single, collaborative platform, 1 2025. URL <https://github.com/>.